不同情况下“板凳龙”的运动分析

摘要

本文主要研究了“板凳龙”的运动问题，建立了“板凳龙”各个把手的运动关系模型和板凳间的运动碰撞模型，同时考虑到“板凳龙”的调头情景，解出了不同情况下“板凳龙”的运动状态。

对于问题一, 建立了把手的**运动关系模型**。首先建立极坐标系，通过**等距螺线方程**求解出了把手的运动距离与角度变化的关系。接着根据运动距离与角度变化的关系，求出了任意时刻的龙头位置。其次，考虑到板凳的**刚性假设**，利用**余弦定理**得到了相邻板凳的位置递推关系，并利用数值求解的方式，求出了任意时刻所有把手的位置。由于板凳的刚性假设，任意两相邻把手在板凳方向的分速度相同，并以此求解出了把手在等距螺线运动的条件下，各个把手的速度的递推关系。最终求出的结果如表 1和表 2所示。

对于问题二, 建立了板凳间的**碰撞模型**。本问采用 **OBB 与 SAT 的结合算法**来作为板凳的碰撞检测算法，同时讨论了“板凳龙”哪些部分会发生碰撞，以及相邻板凳的搜索方式。建立好板凳的碰撞模型后，结合其运动模型，以 0.1s 为步长，每前进 0.1s 检测是否发生碰撞，得到了发生碰撞的时间为 **413s**，该时刻“板凳龙”部分把手的位置和速度如表 3所示。

对于问题三, 建立了基于**拟合分析**和**二分法**的优化模型。首先在任意螺距 δ 的条件下，结合问题一、二建立的把手运动关系模型和板凳的碰撞模型，计算出最终碰撞时龙头的位置 l，将 l 和 δ 进行拟合处理，发现当 δ 位于 $[0.42, 0.5]$ 时,l 是单调函数，因此在这个范围内利用二分法求解龙头的位置 l。基于上诉优化策略，求出了龙头恰好能够调头区域的最小螺距 δ：**0.44972m**

对于问题四, 建立了“板凳龙”的**调头模型**。在调头区域内，“板凳龙”的调头曲线分为盘入螺线部分、圆弧部分、盘出螺线部分。由于大圆弧、小圆弧、盘入螺线、盘出螺线之间存在许多约束，根据这些约束可以计算出**调头路径的长度与调头路径起始点 AB 的关系**。由于问题四中的调头曲线是由多种不同的曲线拼接而成的，本问修正了把手的运动模型，同时还综合考虑了**盘出路径堵塞**、**龙头卡死**、**板凳碰撞**等约束条件，进一步检验调头路径的合理性。最后利用**贝叶斯优化**得到最短调头曲线长度为:**10.11m**

对于问题五, 首先设定龙头的初始速度，利用问题四最终的优化结果和修正后的把手运动模型，求解出各个时刻把手的位置和速度，检验其最大速度是否超过 $2m/s$。对于该最大最小化问题，本问利用**等步长迭代算法**，得到了龙头的最大运动速度为：**1.6m/s**

关键字：等距螺线方程　OBB 与 SAT 结合算法　二分法　贝叶斯优化　等步长迭代算法

一、 问题重述

1.1 问题背景

千米长龙“舞动”六百年，在这项非遗传统“板凳龙”表演中，队员们通过控制连接成一串的板凳，模拟龙的盘旋、翻滚等动作，为观众呈现一场视觉盛宴。为了提高表演的观赏性和安全性，同时也便于更好地将这项优秀的非物质文化遗产传承，对舞龙队的运动进行数学建模和优化分析显得尤为重要。

1.2 问题要求

问题 1 考虑在给定舞龙队的盘入路径，龙头前把手行进速度固定，龙头初始时刻时位置已知的情况下，求解到指定时刻为止，每秒钟整个队列的位置和速度，记录并保存结果。

问题 2 要求在沿着问题一给定的路径盘入，并保证舞龙队列间板凳不发生碰撞的情况下，确定不能再继续盘入的临界时刻，同时求解出此刻舞龙队位置和速度。

问题 3 给定了圆形调头空间的直径，要求在保证龙头把手能够进入掉头空间的情况下，尽可能减小运行轨迹的螺距。

问题 4 本题需要队列在问题 3 的给定的调头空间内，按照本题指定 S 形曲线路径完成调头，要求分析并确定是否可以通过调整圆弧半径，使得调头路径在保持与盘入、盘出螺线相切的条件下变得更短。同时考虑在龙头前把手行进速度给定的情况下，在特定时间区间内求解每秒舞龙队的位置和速度。

问题 5 在遵循问题 4 设定的调头路径和速度的前提下，确定龙头的最大行进速度，以保证舞龙队各把手的速度不超过 2 m/s。

二、 问题分析

2.1 问题一分析

对于问题一，要求我们计算任意时刻所有把手的位置和速度。首先，为了描述把手的运动状态，本文采用极坐标系来构建等距螺旋线模型，并在极坐标系下描述各个把手的位置。为了后续计算，还需要先计算出每个时刻把手的位置，利用积分公式计算等距螺线运动距离和角度变化的关系，并以此得到了每个时刻龙头的位置。其次，利用余弦定理表示任意两个把手之间的距离，而这个距离为已知的定值，这样就得到了相邻把手位置的递推关系，利用龙头位置这个初始值，即可得到任意时刻所有把手的位置。每个

把手的速度方向、大小均不相同，但由于板凳的刚性假设，各个把手的在板凳的分速度均相同，这样就该得到了相邻把手的速度递推公式，利用龙头速度这个初始值，即可得到任意时刻所有把手的速度。

2.2 问题二分析

对于问题二，要求我们在考虑板凳碰撞的情况下，计算“板凳龙”的最大运动时间。为了解决这个问题，首先需要描述什么情况下会发生碰撞，本文采用分离轴定理（SAT）和方向包围盒算法（OBB）的结合算法来检测板凳是否碰撞：将两个矩形的短边作为分离轴，检查两个板凳在分离轴上的投影是否发生重叠。其次，根据等距螺线的结构可以知道，如果发生碰撞，一定是靠内的相邻两层板凳之间发生碰撞，特别是最内层，因此只需要判定最内圈板凳是否会与外圈发生碰撞。然后，设计一个角度最邻近算法，筛选得到每一个待检测板凳可能发生碰撞的板凳。最后，使用 SAT 和 OBB 的结合算法检测这两个板凳是否发生碰撞，一旦在某一时刻检测到碰撞，该时刻即为龙队的盘入终止时刻。再利用与问题一类似的算法，即可给出该时刻所有把手的位置与运动速度。

2.3 问题三分析

对于问题三，要求我们计算出一个最小螺距，能够使龙头前把手恰好可以进入调头空间。针对这个问题，首先需要综合考虑把手的运动模型和板凳碰撞检测模型，计算出“板凳龙”最终停止时龙头距离圆心的位置，然后判断龙头是否进入调头空间。最后可以利用二分法的优化策略，调节不同的螺距，寻找符合题目要求的最小螺距。

2.4 问题四分析

对于问题四，要求我们计算“板凳龙”的最短调头曲线。首先需要讨论调头曲线的几何关系，利用大圆弧、小圆弧、盘出螺线、盘入螺线的相切关系，通过一些列的数学运算可以知道，只要确定了调头曲线的起始点 AB，且该调头曲线是合理的，即可将该曲线确定，利用数学关系，可以算出调头曲线的长度。针对什么样的调头曲线是合理的，认为满足这三个判定条件的调头曲线是合理的：盘出路径是否堵塞、龙头是否卡死、板凳是否碰撞。利用这三个判定条件，可以得到一系列合理的调头曲线起始点 AB，并利用前面推导的公式，计算这条调头曲线的长度。其次，考虑到本问的运动路径是由多端曲线组成的，还需要修正把手的运动模型来计算所有把手的位置和速度。最后，考虑到单次求解时间过长，使用常规优化算法的效率地下，这里可以使用贝叶斯优化，寻找最短的调头曲线，并利用修正后的把手运动模型计算其中某些时刻把手的位置和速度。

2.5 问题五分析

对于问题五，“板凳龙”在问题 4 设定的路径上运动，要求计算龙头的最大速度，使得任意时刻所有把手的速度均不超过 $2m/s$。这里可以利用等步长迭代算法来寻找龙头的最大速度，首先假定一个龙头的初始速度，然后利用问题四中修正过的把手运动模型计算整个调头过程的最大把手速度 V_{max}，如果 $V_{max} > 2m/s$ 则减小龙头的速度，再次重复上诉过程，直到符合题意，计算出龙头的最大速度。

三、 模型假设

为简化问题，本文做出以下假设：

- 假设 1：板凳为一刚体，不考虑它的形变。
- 假设 2：不考虑板凳的厚度和高度变化，仅考虑其在二维平面的干涉情况。

四、 符号说明

符号	说明	单位
λ	螺线常数	
δ	螺距	m
l_{min}	碰撞时龙头位置	m
l_d	龙头卡死距离	m
R	调头空间小圆半径	m
φ	螺旋的切线和位矢的夹角	rad
D	调头区域的直径	m
R	小圆弧半径	m
S	调头曲线长度	m
ξ_{big}	大圆弧的弧长	m
ξ_{small}	小圆弧的弧长	m
$\overrightarrow{n_1}$	调头开始点的法向量	
$\overrightarrow{n_2}$	调头结束点的法向量	

五、 问题一模型的建立和求解

5.1 模型建立

为了计算各个时刻所有把手的位置和速度，首先需要计算各个时刻龙头的位置，然后利用每个板凳的刚性假设来计算相邻把手的位置关系和速率关系递推式，利用龙头的位置和速度这个边界条件即可得出答案。

5.1.1 等距螺旋线模型

任意一条通过原点的直线与螺旋线的交点之间的距离均相等，且间距称为螺距（记为 δ），因此给出等距螺旋线在极坐标下的方程：

$$r = \lambda\theta \tag{1}$$

螺距即是每当 θ 变化 2π 时，r 的变化量，因此：

$$\delta = \lambda \times 2\pi$$

解得：

$$\lambda = \frac{\delta}{2\pi}$$

因此得到本问的等距螺旋线方程为：

$$r = \frac{\delta}{2\pi}\theta \tag{2}$$

可视化图像如下：

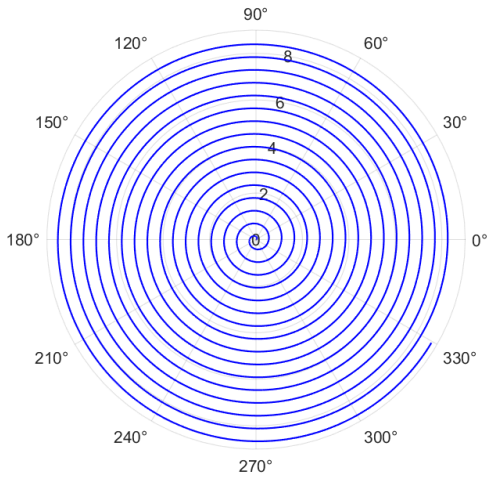


图 1　螺线方程示意图

5.1.2 计算龙头位置

得到把手的运动轨迹后，便可计算每个时刻的龙头位置了。

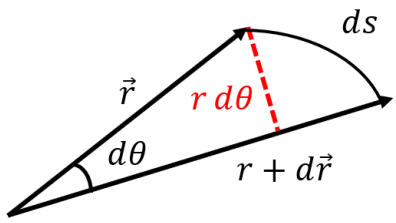


图 2　极坐标 ds 微元示意图

如图，可以计算该等距螺旋线的弧长的微元为：

$$\mathrm{d}s=\sqrt{(\mathrm{d}r)^2+(r\mathrm{d}\theta)^2}=\sqrt{r^2+\lambda^2}\mathrm{d}\theta=\lambda\sqrt{1+\theta^2}\mathrm{d}\theta \tag{3}$$

两边同时积分，即可计算当龙头沿螺线移动 1m（亦即运动 1 秒）后变化的角度，具体的计算公式如下：

$$\int_{\theta_i}^{\theta_{i+1}}\lambda\sqrt{\theta^2+1}\mathrm{d}\theta=\int_0^1\mathrm{d}s=1 \tag{4}$$

求解这个方程，可以得到一个关于 θ_i 和 θ_{i+1} 的隐函数，即：

$$\frac{1}{2}\lambda\left(\ln\left(\theta+\sqrt{1+\theta^2}\right)+\theta\sqrt{1+\theta^2}\right)\Big|_{\theta_i}^{\theta_{i+1}}=1 \tag{5}$$

对于该隐函数而言，显式递推式难以写出，但可以进行数值求解，得出每个时刻的龙头角度。

得到了每个时刻的龙头角度之后，代入 (2) 即可得到对应的极坐标。

5.1.3 相邻把手位置关系

得到了龙头每个时刻的位置之后，便可计算相邻把手之间的位置关系。

考虑到板凳是一个不可形变的刚体，而且知道任意两相邻把手之间的距离 l, 因此可以利用余弦定理计算相邻把手的角度关系：

$$l^2=r_i^2+r_{i+1}^2-2r_ir_{i+1}\cos\left(\theta_i-\theta_{i+1}\right) \tag{6}$$

对于该隐函数，难以写出显式递推式，但是可以进行数值求解，因此仅需要利用公式 (5), 即可得到其余把手的角度位置。

5.1.4 相邻把手速度关系

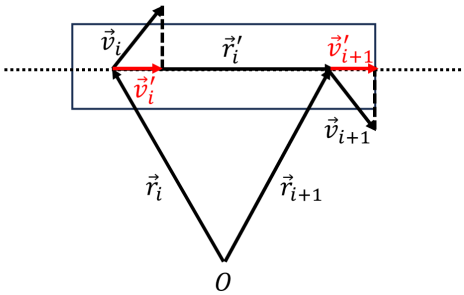


图 3　相邻把手速度关系示意图

如图所示，考虑到板凳是一个不可形变的刚体，因此任意两相邻把手在板凳上的分速度相同，利用该关系即可计算出相邻把手的速度关系，下面求解其具体关系。

首先给出边界条件，即龙头的速度表达式：

$$v_1 = \lambda\dot{\theta}_1\sqrt{1+\theta_1^2} = 1$$

将把手位置从极坐标系转到直角坐标系，有：

$$\begin{cases} x_i = r_i\cos\theta_i \\ y_i = r_i\sin\theta_i \end{cases} \Rightarrow \overrightarrow{r_i} = \begin{bmatrix} r_i\cos\theta_i \\ r_i\sin\theta_i \end{bmatrix} \tag{7}$$

$$\begin{cases} x_i = r_{i+1}\cos\theta_{i+1} \\ y_i = r_{i+1}\sin\theta_{i+1} \end{cases} \Rightarrow \overrightarrow{r}_{i+1} = \begin{bmatrix} r_{i+1}\cos\theta_{i+1} \\ r_{i+1}\sin\theta_{i+1} \end{bmatrix} \tag{8}$$

利用两个把手的位置，计算板凳的方向向量，即：

$$\overrightarrow{r_i}' = \begin{bmatrix} r_i\cos\theta_i - r_{i+1}\cos\theta_{i+1} \\ r_i\sin\theta_i - r_{i+1}\sin\theta_{i+1} \end{bmatrix} \tag{9}$$

对第 i 个把手的位置求导，即可得到它的速度矢量：

$$\begin{cases} v_{ix} = \dot{r}_i\cos\theta_i - r_i\sin\theta_i\dot{\theta}_i \\ v_{iy} = \dot{r}_i\sin\theta_i - r_i\cos\theta_i\dot{\theta}_i \end{cases} \Rightarrow \overrightarrow{v_i} = \begin{bmatrix} \dot{r}_i\cos\theta_i - r_i\sin\theta_i\dot{\theta}_i \\ \dot{r}_i\sin\theta_i - r_i\cos\theta_i\dot{\theta}_i \end{bmatrix} \tag{10}$$

同理可得第 i+1 个把手的速度矢量为：

$$\overrightarrow{v}_{i+1} = \begin{bmatrix} \dot{r}_{i+1}\cos\theta_{i+1} - r_{i+1}\sin\theta_{i+1}\dot{\theta}_{i+1} \\ \dot{r}_{i+1}\sin\theta_{i+1} - r_{i+1}\cos\theta_{i+1}\dot{\theta}_{i+1} \end{bmatrix} \tag{11}$$

两个把手在板凳方向的分速度相同，可以利用点乘来实现，即：

$$\overrightarrow{v_i} \cdot \overrightarrow{r_i}' = \overrightarrow{v}_{i+1} \cdot \overrightarrow{r_i}' \tag{12}$$

将 (8)、(9)、(10) 带入 (11)，有：

$$\begin{bmatrix} \dot{r}_i \cos\theta_i - r_i \sin\theta_i \dot{\theta}_i \\ \dot{r}_i \sin\theta_i - r_i \cos\theta_i \dot{\theta}_i \end{bmatrix} \cdot \begin{bmatrix} r_i \cos\theta_i \\ r_i \sin\theta_i \end{bmatrix} = \begin{bmatrix} \dot{r}_{i+1} \cos\theta_{i+1} - r_{i+1} \sin\theta_{i+1} \dot{\theta}_{i+1} \\ \dot{r}_{i+1} \sin\theta_{i+1} - r_{i+1} \cos\theta_{i+1} \dot{\theta}_{i+1} \end{bmatrix} \cdot \begin{bmatrix} r_i \cos\theta_i - r_{i+1} \cos\theta_{i+1} \\ r_i \sin\theta_i - r_{i+1} \sin\theta_{i+1} \end{bmatrix}$$

化简得到：

$$\dot{\theta}_{i+1} = \frac{\begin{vmatrix} r_i \cos\theta_i - r_{i+1} \cos\theta_{i+1} & \lambda \cos\theta_i - r_i \sin\theta_i \\ r_i \sin\theta_i - r_{i+1} \sin\theta_{i+1} & -r_i \cos\theta_i - \lambda \sin\theta_i \end{vmatrix}}{\begin{vmatrix} r_i \sin\theta_i - r_{i+1} \sin\theta_{i+1} & \lambda \sin\theta_{i+1} + r_{i+1} \cos\theta_{i+1} \\ r_i \cos\theta_i - r_{i+1} \cos\theta_{i+1} & r_{i+1} \sin\theta_{i+1} + \lambda \cos\theta_{i+1} \end{vmatrix}} \dot{\theta}_i \tag{13}$$

至此，得到了任意两相邻把手的角速度递推关系，将龙头速度这个边界条件和任意时刻所有把手的位置带入，即可求解得到所有把手在任意时刻的角速度。

在螺线部分，把手的速度可以表示为：

$$v = \lambda \dot{\theta} \sqrt{1 + \theta^2}$$

5.2 模型求解

Step1：计算各个时刻的龙头位置

利用公式 (5)，并带入龙头的初始位置，即可计算每个时刻的龙头位置角度。将龙头位置角度带入公式 (2), 即可得到龙头的完整位置。

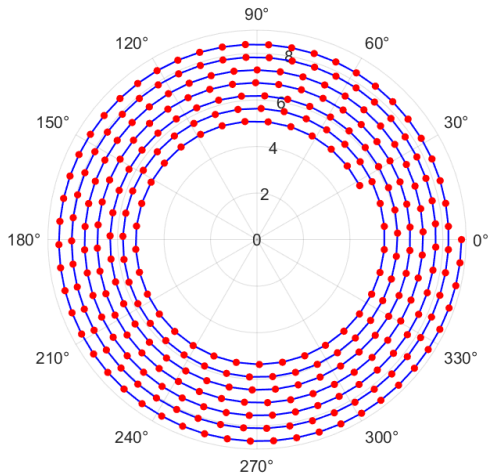


图 4　龙头位置

图 4是在 300s 内每一秒的龙头位置的可视图。其中，红色圆点为龙头在各个时刻的位置，蓝色曲线为龙头的运动轨迹线。

Step2：计算所有把手位置

相邻把手的位置关系由公式 (5) 给出，但是这个函数是关于 θ_i 和 θ_{i+1} 的隐函数，无法求出其对应的解析解，因此需要利用 matlab 求解这个隐函数，对应代码见附录。利用该递推方程和每个时刻的龙头位置，即可求出每个时刻的所有把手的位置。

Step3：计算所有把手速度

此时，我们已经得到了每个时刻所有把手的位置，再利用公式 (11)，带入初始条件龙头速度，即可求解出每一时刻所有把手的速度。

5.3 求解结果

问题一位置结果和速度结果如表 1和表 2所示

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 x (m)	8.800000	5.799209	-4.084887	-2.963609	2.594494	4.420274
龙头 y (m)	0.000000	-5.771092	-6.304479	6.094780	-5.356743	2.320429
第 1 节龙身 x (m)	8.363824	7.456758	-1.445473	-5.237118	4.821221	2.459489
第 1 节龙身 y (m)	2.826544	-3.440399	-7.405883	4.359627	-3.561949	4.402476
第 51 节龙身 x (m)	-9.518732	-8.686317	-5.543149	2.890456	5.980011	-6.301346
第 51 节龙身 y (m)	1.341137	2.540108	6.377946	7.249289	-3.827759	0.465830
第 101 节龙身 x (m)	2.913983	5.687115	5.361939	1.898794	-4.917372	-6.237721
第 101 节龙身 y (m)	-9.918311	-8.001384	-7.557638	-8.471614	-6.379874	3.936009
第 151 节龙身 x (m)	10.861726	6.682312	2.388757	1.005154	2.965379	7.040741
第 151 节龙身 y (m)	1.828753	8.134544	9.727411	9.424751	8.399720	4.393012
第 201 节龙身 x (m)	4.555102	-6.619663	-10.627210	-9.287720	-7.457152	-7.458663
第 201 节龙身 y (m)	10.725118	9.025570	1.359847	-4.246672	-6.180725	-5.263383
龙尾（后）x (m)	-5.305444	7.364557	10.974348	7.383896	3.241052	1.785035
龙尾（后）y (m)	-10.676584	-8.797992	0.843473	7.492370	9.469336	9.301164

表 1　问题一位置结果

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999971	0.999961	0.999945	0.999917	0.999859	0.999709
第 51 节龙身 (m/s)	0.999742	0.999662	0.999538	0.999331	0.998941	0.998065
第 101 节龙身 (m/s)	0.999575	0.999453	0.999269	0.998971	0.998435	0.997302
第 151 节龙身 (m/s)	0.999448	0.999299	0.999078	0.998727	0.998115	0.996861
第 201 节龙身 (m/s)	0.999348	0.999180	0.998935	0.998551	0.997894	0.996574

表 2　问题一速度结果

将龙头运动了 300s 后的把手位置在图上画出，可以得到 60s 时，板凳龙速度的可视化图像：

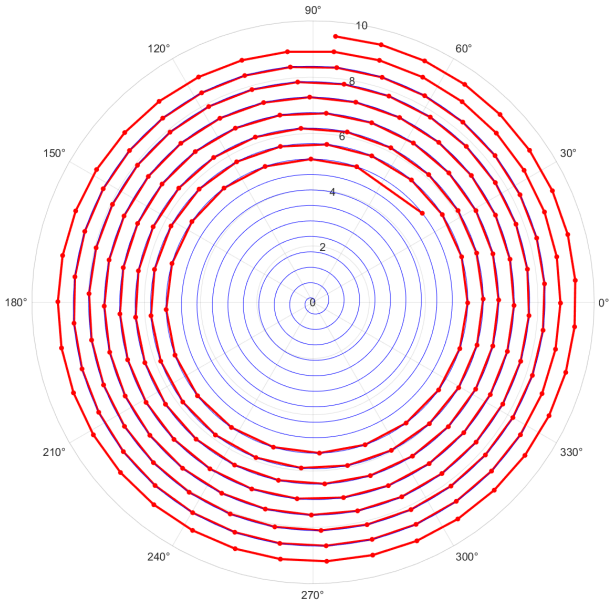


图 5　300s 时板凳龙的位置示意图

如图 5，红色的线条是板凳，红色线条交点是各个把手.

5.4 结果分析

如表 2所示，可以发现当“板凳龙”沿着等距螺线盘入时，最内层龙头把手的速度是最大的，随着层数的外推，各个把手的速度逐渐减小，这个结论对第五的求解有一定的作用。

六、 问题二模型的建立和求解

6.1 模型建立

问题二要求考虑板凳之间的碰撞，计算什么时候不能进一步盘入，同时计算出此时刻舞龙队位置和速度。因此首先需要建立碰撞检查模型，然后再讨论可能产生碰撞的区域来减少运算量，接下来将分别讨论这两个模型。

6.1.1 碰撞区域讨论

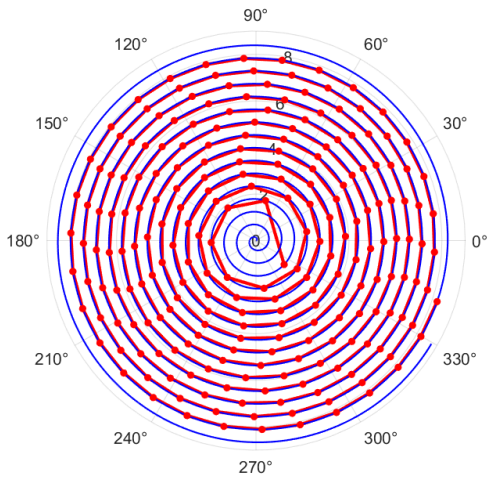


图 6　某时刻“板凳龙”示意图：图中红点为把手位置

如图 6所示，给出了某一时刻板凳龙的可视化图像，可以发现越往中间走，板凳越出螺旋线的距离越大，越可能与外环的板凳发生干涉。因此，首先发生干涉的板凳应该是最靠近原点的一环上的板凳。于是，只需要检测最内环板凳上是否与外环板凳发生干涉即可。

其次，再讨论哪些相邻板凳会发生碰撞。

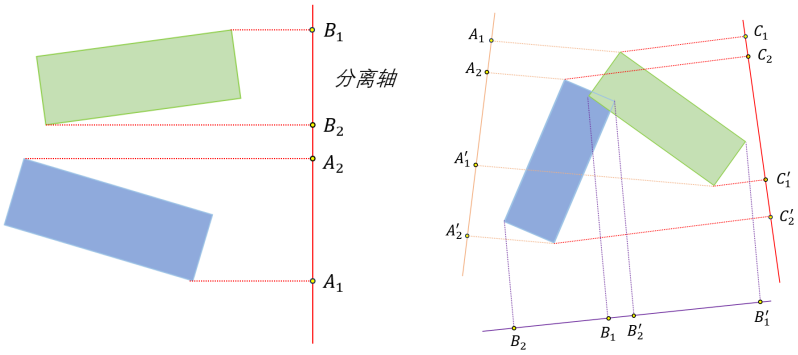
由上述讨论可知，发生碰撞的情况肯定是内环与外环的板凳发生碰撞，碰撞的部位是内环的顶角和外环的边。考虑到板凳位置的特殊结构，这里假设需要检验的板凳的前把手的位置是 θ，则只需检测距离 $\theta + 2\pi$ 最近的两个把手组成的板凳与该板凳是否发生碰撞即可。

6.1.2 碰撞检测模型

由文献 [1] 可知，对于碰撞检查而言，目前有多种方法，包括：像素检测算法、包围球算法、沿坐标轴包围算法（AABB）和方向包围盒算法（OBB）等等。考虑到计算量和本文的精确度，这里选择 OBB 算法和分离轴定理（SAT）的结合算法。

分离轴定理 SAT

首先介绍什么是分离轴定理：如果能找到一条轴，使得两个物体在该轴上的投影不重叠，那么这两个物体就是不相交的。如下图所示：



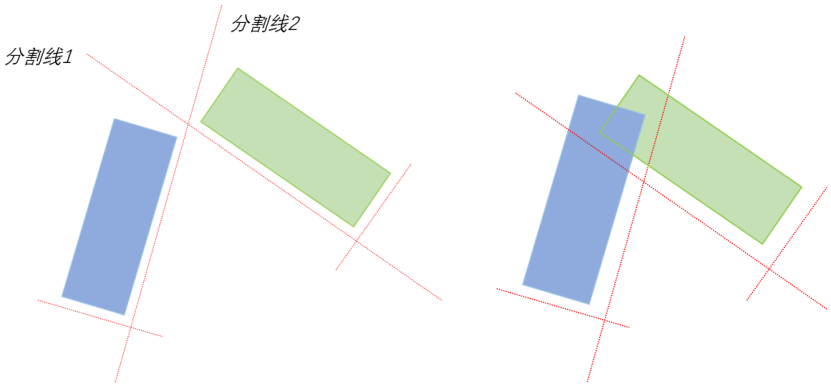
(a) SAT 分离情况 **(b) SAT 碰撞情况**

图 7　SAT 示意图

可是分离轴定理有一个明显的缺点：可行的分离轴很难找到。如果任意做一条直线然后 360 度旋转求解的话，计算量又过大。

方向包围盒算法 OBB

现在介绍一下方向包围盒算法：若两个矩形不碰撞，那么必然存在一条与两个矩阵的某一边平行的分割线，这条分割线可以将两个举行分开。如图 8所示：



(a) OBB 分离情况 **(b) OBB 碰撞情况**

图 8　OBB 示意图

由于计算分割线是需要先把每一个边向外平移一定距离，而且还需要判定方向，最后还要计算两个矩阵是否位于分割线的两侧，因此 OBB 算法也有一个明显的缺点：这个分割线的具体位置的求解需要消耗一定的算力。

OBB 与 SAT 的结合算法

为了消除 OBB 算法和 SAT 算法的缺点，本文采用 OBB 和 STA 的结合算法。OBB 算法和分离轴定理结合起来可以很好的判断任意两个矩形是否相交，具体的算法原理如下：

由 OBB 算法可以知道，若两矩形不碰撞，则一定存在某一条与矩形边平行的分割线，因此分割线的垂线与矩形对应的边的垂线是平行的，而考虑到矩形的两边是相互垂直的，因此分割线的垂线也与矩形某一条边平行。此外，如图 9所示，分割线的垂线即是 SAT 算法中的分离轴，因此在两个不相碰撞的矩形中，一定存在一个与矩形边平行的分离轴。

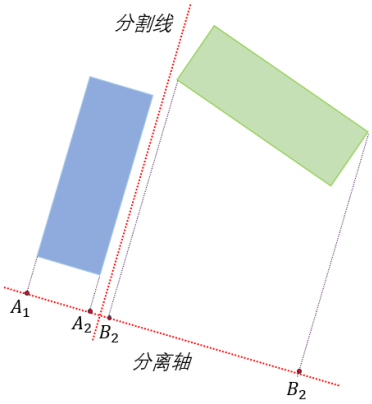


图 9　OBB 和 STA 的结合算法示意图

根据上诉算法原理，可以利用下面的步骤来判断两个矩形是否碰撞：

step1：选择可能发生碰撞的两个板凳，并且提取它们的四个顶角的位置。

step2：任意选择一个矩阵，计算宽方向的方向向量。

step3：将它们的四个顶角投影到这个方向向量上。

step4：判断四个投影点是否发生重叠现象。如果没有重叠现象，则证明这两个板凳没有发生碰撞；如果发生重叠现象，则证明这两个板凳发生了碰撞。

6.2 模型求解

Step1：选取合适的开始时刻

初始时以 0 时刻为开始时刻，检测代码层面是否存在数组越界错误。出现这种错误的原因是：在较小的开始时刻，龙队相邻两层邻近点之间间隔的把手数目过多，并且难以通过简单的计算来预测这种间隔的变化。由于此时龙队位于非常外层的位置，因此产生数组越界错误时认为整个龙队不存在碰撞。

在检测到数组越界错误后，可以直接将开始时刻增大 50s，然后再进行检测，如此往复直到数组越界错误消失为止。此时的开始时刻即为合适的开始时刻。

Step2：细分搜索碰撞时刻

利用问题一建立的模型，从合适的开始时刻开始，以 1s 为步长，逐步求解所有时刻的把手的位置并通过检测算法检测内层前 50 个点与其对应外层之间是否发生碰撞。

Step3：结果判断

如果龙头卡死（极限终止时刻）之前就检测出了碰撞，则该时刻则为大致的终止时刻，代码层面表现为终止时刻为一个较大的正数（大于 1）。如果代码层面终止时刻等于 1，则说明在龙头卡死之前都没有发生碰撞，也就是说不会发生碰撞。

Step4：更精细的结果求解

在原精确度的基础上，选用更加精细的时间搜索步长，重复上述流程即可。

上述求解过程的流程图如下：

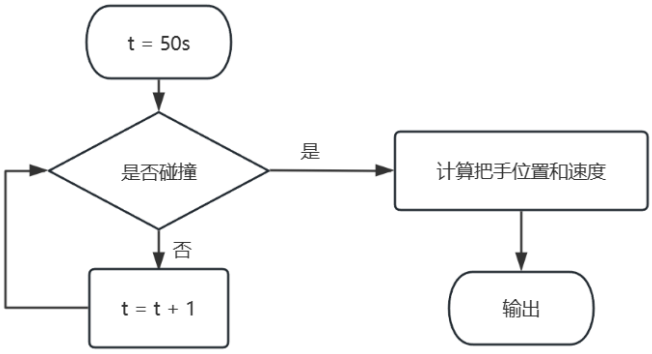


图 10　问题二流程图

6.3 求解结果

最终结果如表 3所示

	横坐标 x (m)	纵坐标 y (m)	速度 (m/s)
龙头	0.793016	2.166186	1
第 1 节龙身	-1.965804	1.412191	0.991858
第 51 节龙身	0.833481	4.443781	0.977434
第 101 节龙身	-0.075475	-5.910900	0.975140
第 151 节龙身	1.424424	-6.884545	0.974201
第 201 节龙身	-7.813964	-1.685074	0.973698
龙尾（后）	0.497306	8.367538	0.973532

表 3　第二问求解结果

七、问题三模型的建立和求解

7.1 模型建立

本问要求在保证“板凳龙”能够顺利进入掉头空间的情况下使得螺距最小化。针对这个问题，需要综合利用问题一二中建立的把手运动模型和板凳碰撞检测模型，计算某一螺距下对应的最终停止时的龙头位置，然后判断龙头是否进入调头空间即可。

因此建立如下的优化模型

目标函数：

$$min\left\{\delta\right\}$$

约束条件：

$$s.t.\begin{cases} l_{min} \leq \frac{D}{2} \\ D = 9 \\ l^2 = r_i^2 + r_{i+1}^2 - 2r_i r_{i+1}\cos\left(\theta_i - \theta_{i+1}\right) \\ \dot{\theta}_{i+1} = \dfrac{\begin{vmatrix} r_i\cos\theta_i - r_{i+1}\cos\theta_{i+1} & \lambda\cos\theta_i - r_i\sin\theta_i \\ r_i\sin\theta_i - r_{i+1}\sin\theta_{i+1} & -r_i\cos\theta_i - \lambda\sin\theta_i \end{vmatrix}}{\begin{vmatrix} r_i\sin\theta_i - r_{i+1}\sin\theta_{i+1} & \lambda\sin\theta_{i+1} + r_{i+1}\cos\theta_{i+1} \\ r_i\cos\theta_i - r_{i+1}\cos\theta_{i+1} & r_{i+1}\sin\theta_{i+1} + \lambda\cos\theta_{i+1} \end{vmatrix}}\dot{\theta}_i \end{cases}$$

其中 l_{min} 为板凳之间无碰撞的极限时刻，D 为题目所给掉头区域的直径, 由于“板凳龙”是在等距螺线上运动，因此相邻把手的速度和位置满足那两个递推式。

决策变量：

本问的决策变量只有一个，即螺距 δ，同时也是待优化的目标函数。考虑到问题的特殊性（单变量和单调性，后文有具体说明），该优化问题可以退化为函数零点的求解问题，利用拟合算法和二分法即可求出最终答案。

7.2 模型求解

Step1：初步筛选合适的范围

表 4　采样范围

δ	0.4	0.5
l_{min}	6.2997	3.0944

这一步可以随机选取 2 个的螺距数据，例如表 4所示：

考虑表 4中 l_{min} 与 $\frac{D}{2}$ 的关系，不妨将 δ 的取值范围限定在区间：$[0.4, 0.5]$，并在这个小区间内进行细分采样，更细致地研究 δ 与 l_{min} 之间的函数关系。

Step2：密集采样验证预测

在区间 $[0.4, 0.5]$ 中，等距取采样 30 个采样点并计算目标函数即 l_{min} 的值，然后利用这些值进行多项式的拟合。拟合过程中需要注意多项式的次数选择，经过测试可以发现 5 次多项式既描述了原点列的细节变化趋势，又不至于过度拟合。最终拟合图形如下：

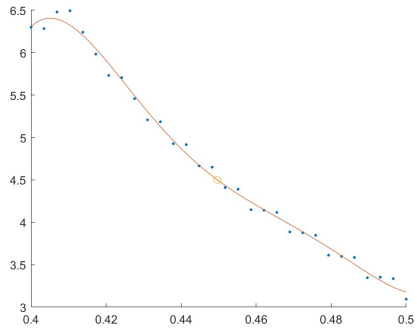


图 11　问题三拟合结果

通过图 11可以得出结论：在区间 $[0.42, 0.5]$ 上，函数 $g(\delta) = l_{min}(\delta) - \frac{D}{2}$ 有唯一零点。因此本问中的优化问题退化为函数零点的求解问题。

Step3：求根算法

考虑到本问的相关函数 $g(x)$ 无法得到其解析式，只能访问其函数值，但是能够确定其零点唯一，简单起见，本问直接利用二分法求解相关函数的零点。

7.3 求解结果

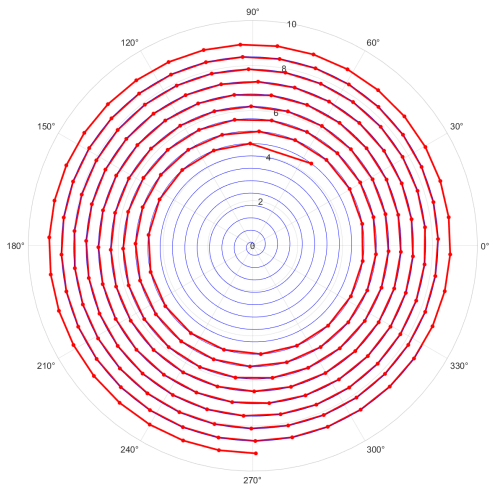


图 12　龙头恰好进入调头空间的示意图

龙头恰好进入掉头空间的情况如图 12所示，其中，该等距螺线对应的螺距 $\delta_{min} = 0.449720$。

7.4 结果分析

从拟合结果来看，l_{min} 关于 δ 的函数在区间 $[0.42, 0.5]$ 中呈现单调递减，符合直观上的逻辑：螺距越小，板凳之间碰撞的可能性越大，龙头停止时离圆心距离越远。但是，同时也发现其在区间 $[0.4, 0.42]$ 有个峰值，在这个峰值左边，l_{min} 关于 δ 单调递增，这是由于，当螺距变小时，龙头的初始位置也是越靠近圆心的，龙头卡死的位置也会离与圆心越近。

八、 问题四模型的建立和求解

8.1 模型建立

问题四要求计算最短的调头曲线，因此需要建立“板凳龙”的调头模型。为了描述调头模型，需要先讨论调头路径的计算方法。同时，还需要考虑调头过程中的约束关系，包括龙头卡死、盘出路径堵塞和板凳碰撞。此外，考虑到调头部分“板凳龙”是做的圆周运动，还需要对他的运动模型进行修正。最后根据得到的调头模型再确立相应的优化模型。接下来将分别讨论这些内容。

8.1.1 计算调头路径

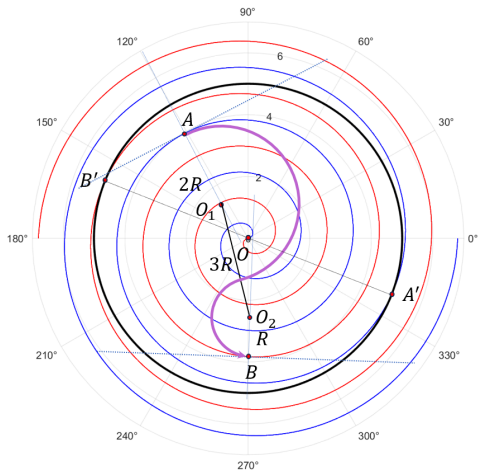


图 13　调头路径示意图

如图 13所示，蓝色螺线为盘入螺线，红色螺线为盘出螺线，黑色圆环为调头空间边界，紫色圆弧为调头路径，A 点为龙头开始调头的位置，B 点为龙头结束调头的位置，O_1 为大圆弧圆心，O_2 为小圆弧圆心，O 为调头空间圆心。

本问对内部调头曲线、盘出螺线、盘入螺线有着许多约束，具体的约束如下：

1. 大圆弧半径是小圆弧的两倍
2. 小圆弧和大圆弧相切
3. 大圆弧与盘入螺线相切
4. 小圆弧与盘出螺线相切
5. 盘出螺线与盘入螺线关于原点 O 中心对称

利用上诉的约束，并给定开始调头的点 A 和结束调头的点 B，可以计算出整个调头轨迹，具体的计算方式如下：

计算 A,B 点的法向方向

由于大圆弧和盘入螺线相切，小圆弧和盘出螺线相切，因此大圆弧和小圆弧的圆心分别在 A,B 点的法线方向，所以求解 A,B 点的法向量是十分重要的。下面推导其法向量的计算公式

该等距螺线的切向量可以表示为：

$$\overrightarrow{T} = \begin{bmatrix} \cos\theta - \theta\sin\theta \\ \sin\theta + \theta\cos\theta \end{bmatrix}$$

根据该切向量可以求出对应的两个方向的法向量，如 N_1 和 N_2 所示：

$$\overrightarrow{N_1} = \begin{bmatrix} -\sin\theta - \theta\cos\theta \\ \cos\theta - \theta\sin\theta \end{bmatrix}$$

$$\overrightarrow{N_2} = \begin{bmatrix} \theta\cos\theta + \sin\theta \\ \theta\sin\theta - \cos\theta \end{bmatrix}$$

由于：

$$N_1 \cdot \overrightarrow{r} = -\lambda\theta\cos\theta\left(\sin\theta + \theta\cos\theta\right) + \lambda\theta\sin\theta\left(\cos\theta - \theta\sin\theta\right) = -\lambda\theta^2 < 0$$

因此，等距螺线的内法向量为：

$$\overrightarrow{N} = \overrightarrow{N_1} = \begin{bmatrix} -\sin\theta - \theta\cos\theta \\ \cos\theta - \theta\sin\theta \end{bmatrix}$$

计算两个圆弧的半径

利用向量加法计算大圆弧和小圆弧的圆心 $\overrightarrow{O}_1$、$\overrightarrow{O}_2$：

$$\overrightarrow{O}_1 = 2R\frac{\overrightarrow{n}_1}{|\overrightarrow{n}_1|} + \overrightarrow{r}_1$$

$$\overrightarrow{O}_2 = R\frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|} + \overrightarrow{r}_2$$

由于大圆弧和小圆弧相切，而且调头曲线是一个“S”型曲线，因此两个圆心的间距为3R，因此：

$$\left|\overrightarrow{O}_1 - \overrightarrow{O}_2\right| = 3R = \left|\overrightarrow{r}_1 - \overrightarrow{r}_2 + R\left(\frac{2\overrightarrow{n}_1}{|\overrightarrow{n}_1|} - \frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|}\right)\right|$$

化简得：

$$aR^2 + bR + c = 0$$

其中：

$$\begin{cases} a = \left(\frac{2\overrightarrow{n}_1}{|\overrightarrow{n}_1|} - \frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|}\right)^2 - 9 \\ b = 2\left(\frac{2\overrightarrow{n}_1}{|\overrightarrow{n}_1|} - \frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|}\right)(\overrightarrow{r}_1 - \overrightarrow{r}_2) \\ c = (\overrightarrow{r}_1 - \overrightarrow{r}_2)^2 \end{cases}$$

考虑到：

$$\begin{cases} R_1R_2 = \frac{c}{a} < 0 \\ b^2 - 4ac - b^2 = -4ac \geq 0 \end{cases}$$

计算得到小圆弧的半径为：

$$R = \frac{b + \sqrt{b^2 - 4ac}}{-2a}$$

计算调头曲线长度

知道了两个圆心的位置后便可以计算调头曲线的长度了，首先根据点乘的定义有：

$$\frac{\overrightarrow{n}_1}{|\overrightarrow{n}_1|} \cdot \frac{\overrightarrow{O_1O_2}}{\left|\overrightarrow{O_1O_2}\right|} = \cos\varphi_1$$

化简得：

$$\varphi_1 = \arccos\left(\frac{\overrightarrow{n}_1}{|\overrightarrow{n}_1|} \cdot \frac{\overrightarrow{O_1O_2}}{\left|\overrightarrow{O_1O_2}\right|}\right)$$

同理得到 φ_2

$$\varphi_2 = \arccos\left(\frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|} \cdot \frac{\overrightarrow{O_1O_2}}{\left|\overrightarrow{O_1O_2}\right|}\right)$$

由于调头曲线可以表示为：

$$S = 2R(\pi + \varphi_1) + R(\pi + \varphi_2)$$

带入 φ_1、φ_2 有：

$$S = 3\pi R + R\left[2\arccos\left(\frac{\overrightarrow{n}_1}{|\overrightarrow{n}_1|}\cdot\frac{\overrightarrow{O_1O_2}}{\left|\overrightarrow{O_1O_2}\right|}\right) + \arccos\left(\frac{\overrightarrow{n}_2}{|\overrightarrow{n}_2|}\cdot\frac{\overrightarrow{O_1O_2}}{\left|\overrightarrow{O_1O_2}\right|}\right)\right]$$

8.1.2 把手运动模型修正

在问题四中，“板凳龙”的运动轨迹发生了变化，现在的运动轨迹难以使用极坐标进行描述，因此本小节将利用自然坐标对运动模型进行修正。

建立自然坐标与直角坐标的映射关系 $\Gamma(s)$

我们选取开始掉头的位置，即 $t=0$ 的时刻龙头所在的位置为自然坐标系的零点，龙头沿调头曲线行进的方向记为坐标 s 的正方向。考虑到题目所给轨迹是分段函数，因此映射关系也应分段求解：1）当 $s<0$ 时，自然坐标实际表示的点位于盘入螺旋线上，可以由公式 (5) 简单求得；2）当 $0<s<\xi_{big}$ 时，ξ_{big} 为大圆总弧长，此时的弧坐标 s 即表示在大圆上行进的弧长，可以由圆的弧长公式简单求解；3）当 $\xi_{big}<s<\xi_{big}+\xi_{small}$ 时，与上一种情况类似，实际坐标可以由圆的弧长公式简单求解；4）当 $\xi_{big}+\xi_{small}<s$ 时，实际表示点位于盘出螺旋线上，同样可由公式 (5) 简单求得；

因此，龙头在直角坐标系中的位置可以表示为，其中 $\Gamma(s)$ 输出为一列向量：

$$\Gamma(s) = \begin{cases} f_1(s) & s<0 \\ f_2(s) & 0<s<\xi_{big} \\ f_3(s) & \xi_{big}<s<\xi_{big}+\xi_{small} \\ f_4(s) & \xi_{big}+\xi_{small}<s \end{cases}, \quad s = v_0 t$$

相邻把手位置关系修正

鉴于曲线的复杂性，直接用直角坐标系中的曲线方程联立求解将会难以判断增根，因此利用自然坐标系来标定曲线上的点尤为重要。考虑第 i 号点的自然坐标 s_i 和第 $i+1$ 号点的自然坐标 s_{i+1} 亦即它们之间的固定长度 L_i，我们可以认为 $s_{i+1} = solve(\Gamma(s) - L_i = 0)$ 距离 s_i 最近的零点。

相邻把手速度关系修正

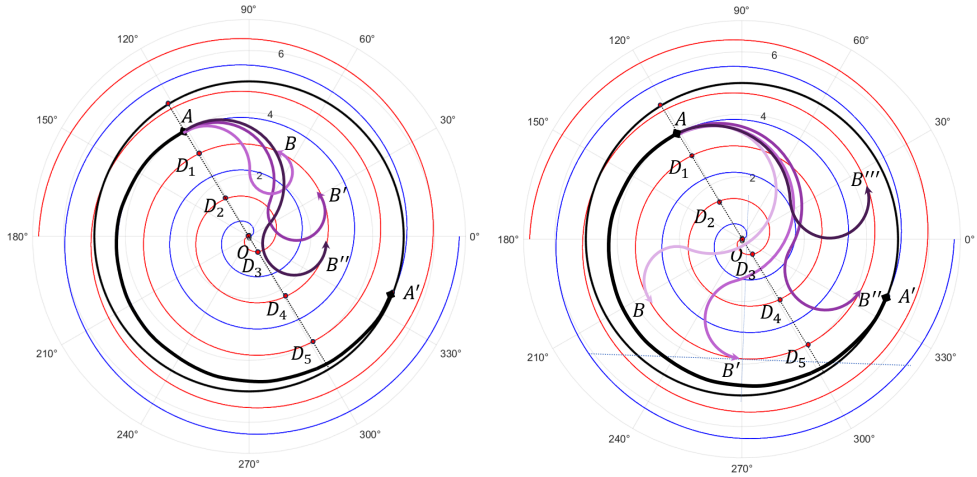
与问题一相同，这里同样是利用了板凳的刚性假设，两个把手在板凳方向的分速度相同。与问题一不同是，为了求解把手的速度方向，需要先判定把手所处位置，在根据对应的曲线来计算他的速度方向。与 $\Gamma(s)$ 的求解类似，我们可以建立自然坐标 s 和单位切向量 $\tau(s)$ 之间的映射关系。

8.1.3 调头路径约束条件

根据上面的讨论，只需要知道调头曲线的起始点 AB 即可求解出整个调头曲线，但是它们并不是可以随机取得的，它们是符合某种要求的，本小节将综合讨论盘出路径堵塞、龙头卡死、板凳碰撞的情况以及它们的判定方法，通过这三个约束条件即可得到合理的调头路径。

盘出路径堵塞

根据题目中给出的约束和前面的讨论，在只考虑三条曲线的相切关系的情况下，可以找到许多满足题意的螺线，但是其中相当一部分会出现盘出路径堵塞的情况。如图 14a所示，龙头从 B 点结束调头后，进入盘出曲线，但是盘出曲线被调头曲线堵住了，因此该调头路径走不通。



(a) 盘出螺线堵塞　　**(b) 盘出螺线无堵塞**

图 14　盘出螺线是否堵塞示意图

现在来研究什么地方不会出现盘出路径堵塞。

如图 14a所示，可以发现，当调头结束点是位于 D_1、D_2、D_3 点内，由于“板凳龙”要到达调头结束点，必然会跨过它下面的那条盘出曲线，而这些调头结束点都是在堵塞点后方，因此都会出现路径堵塞的情况。

如图 14b，可以发现，对于处在这个区域内的调头结束点 B，都不会出现盘出路径堵塞的情况。

综上所示，在盘出螺线中，若调头结束点 B，是在以 A 的中心对称点为中心，上下调节角度小于 π 的区域内，都不会出现盘出路径堵塞的情况，该区域如图 15中的绿色画线部分。

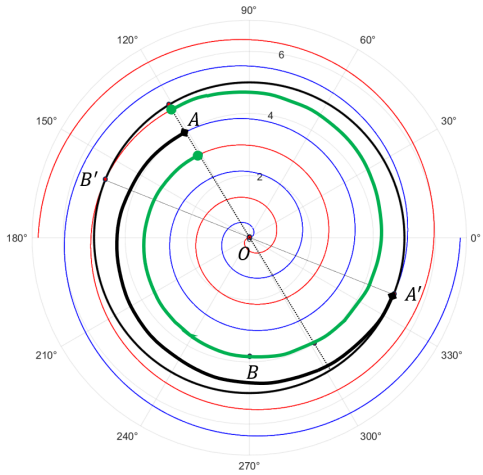
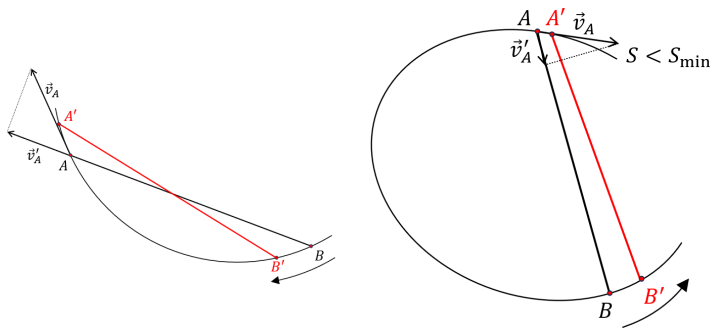


图 15　不会出现堵塞的区域

龙头卡死

对于板凳龙而言，任意位置都是不允许倒退的，考虑到板凳的刚性假设，当龙头沿板凳的分速度是负数时，下一个把手的运动必然是倒退的如图 16所示。这与前面的条件发生矛盾，称这种情况为龙头卡死。在“板凳龙”调头的过程中，由于调头曲线的曲率未知，因此可能出现龙头恰好卡死的情况。



(a) 龙头未卡死　　**(b) 龙头卡死**

图 16　龙头卡死判定图

如果在调头过程中出现龙头卡死的情况，则该 AB 两点不可作为调头曲线的起始点。

板凳碰撞讨论

由问题二可知，对于盘出螺线和盘入螺线部分只需讨论其对应的最内环是否发生碰撞即可。但是考虑到调头区域的复杂性，还需要讨论圆弧曲线部分是否发生碰撞。这里同样使用 SAT 和 OBB 的结合算法来判断板凳是否发生碰撞。

8.2 模型求解

Step1：

在调头空间内，分别在盘出曲线和盘入曲线随机取一个点，作为调头曲线的起始点AB。同时设定贝叶斯优化的相关参数。

Step2：

判断盘出路径是否被调头曲线卡死，如果被卡死，则重新取点，否则进入 Step3。

Step3：

根据前面建立的调头模型，计算调头路径的两个圆心位置，并画出调头曲线，用于Step4 的判断。

Step4：

利用修正后的把手运动模型和 Step3 计算处的调头曲线，计算每个时刻把手的速度和位置，并利用龙头卡死的判定条件来判定该过程是否会卡死，同时利用 OBB 和 SAT 的结合算法检测特定区域的板凳是否发生碰撞。上述的两个约束必须全部满足才能进入Step5，否则重新取点。

Step5：

根据 Step3 求解出的调头曲线，计算调头曲线长度 S，并于之前得到的最短长度 S_{min} 比较，如果 $S < S_{min}$ 则更新 S_{min} 为 S，否则重新取点，重复 Step2~4

Step6：

当循环次数达到设定值时，退出循环，并输出最短调头曲线长度 S_{min}

上述求解过程的流程图如下：

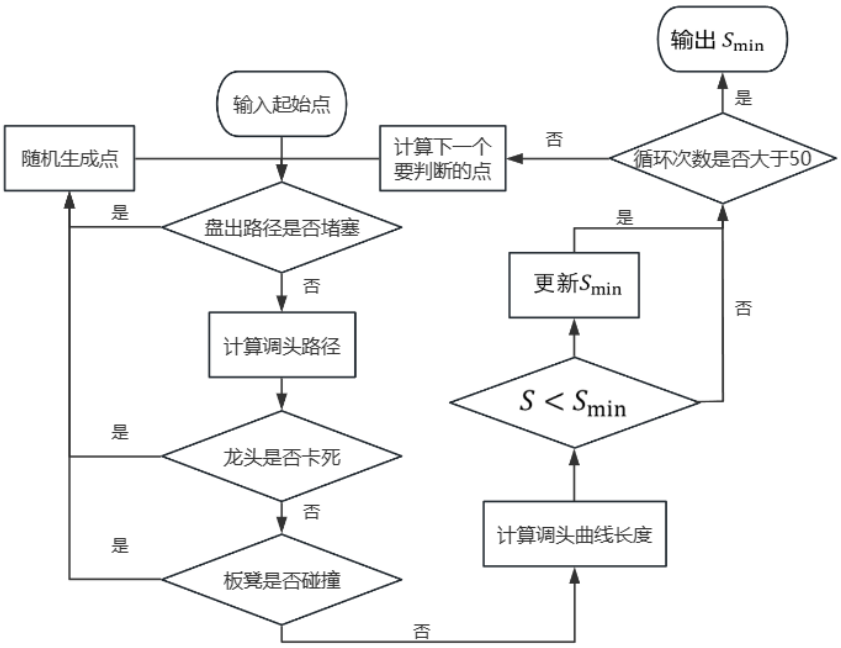


图 17　问题四流程图

8.3 求解结果

图 18是“板凳龙”沿着问题四求解出的调头曲线运动 7s 后的位置，其中黄色折线是“板凳龙”，A 点是调头曲线的开始位置，B 点是调头曲线的结束位置，黑色曲线为调头曲线。

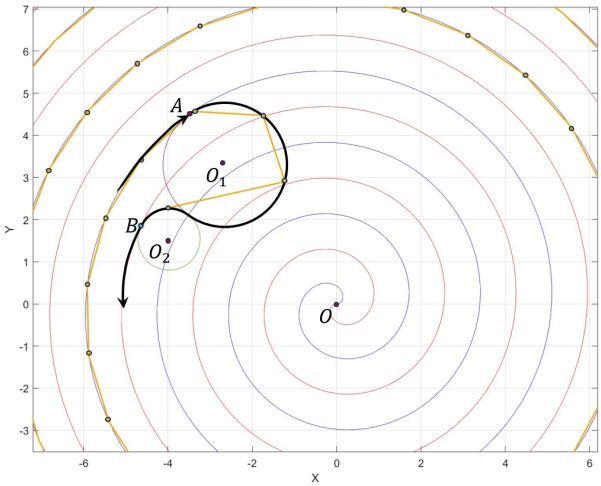


图 18　问题四板凳龙位置示意图

问题四的部分求解结果如表 5和表 2所示

	-100s	-50 s	0 s	50 s	-100 s
龙头 x (m)	-9.182805	-3.693572	-2.187391	-6.117602	-7.350366
龙头 y (m)	1.508829	-4.982710	6.087257	5.608246	-4.560588
第 1 节龙身 x (m)	-9.291378	-1.111632	0.671932	-3.795057	-8.350618
第 1 节龙身 y (m)	-1.350853	-6.228799	6.310628	7.274694	-1.881516
第 51 节龙身 x (m)	1.671405	-8.098666	-7.615837	-1.349522	-3.611495
第 51 节龙身 y (m)	-11.392459	4.397216	0.500291	4.014887	-3.910778
第 101 节龙身 x (m)	6.133503	-4.177267	9.938307	5.795482	7.452219
第 101 节龙身 y (m)	-11.806500	-10.578331	-2.038700	6.526853	-3.815040
第 151 节龙身 x (m)	0.335685	0.372208	7.703145	-10.667271	-3.178934
第 151 节龙身 y (m)	-14.877317	-13.177385	9.381728	2.622004	10.223742
第 201 节龙身 x (m)	-13.672760	-5.4821200	8.396855	-11.998747	-11.005016
第 201 节龙身 y (m)	-8.884725	-13.717222	11.012747	-4.596338	6.160382
龙尾（后）x (m)	15.634020	13.935066	-13.783962	13.576156	8.615248
龙尾（后）y (m)	-6.404068	6.601604	-4.619649	-0.578537	-10.217068

表 5　问题四位置结果

	-100s	-50 s	0 s	50 s	100 s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999929	0.999851	0.999494	1.000247	1.000099
第 51 节龙身 (m/s)	0.999500	0.999060	0.997576	1.161020	1.002619
第 101 节龙身 (m/s)	0.999287	0.998745	0.997053	1.159838	0.862447
第 151 节龙身 (m/s)	0.999159	0.998574	0.996809	1.159413	0.861903
第 201 节龙身 (m/s)	0.999075	0.998466	0.996667	1.159192	0.861667
龙尾（后）(m/s)	0.999045	0.998431	0.996620	1.159125	0.861600

表 6　问题四速度结果

8.4 结果分析

这里随机生成了 50 组符合约束的调头曲线起始点 AB，并且根据问题四建立的调头模型，计算出了对应的调头曲线的长度，将其与问题四的求解结果比较，最终做出的针状图如下：

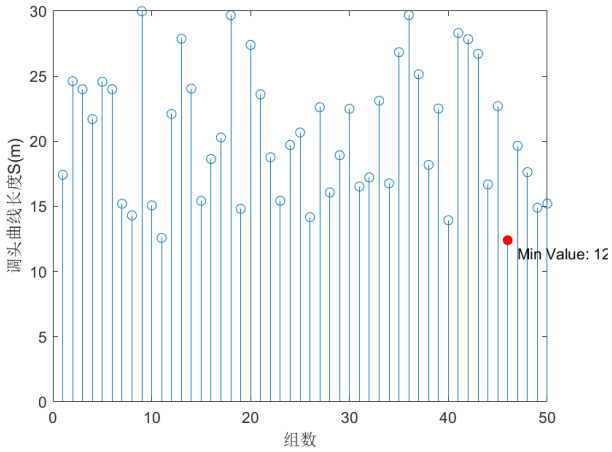


图 19　调头曲线针状图

如图所示，对于这 50 组 AB 点，其对应的调头曲线的长度均大于问题四的求解结果，这进一步证明了模型的正确性和优化策略的合理性。

九、 问题五模型的建立和求解

9.1 模型建立

根据问题四修正过后的把手运动模型和其求解出的运动路径，可以计算出任意时刻所有把手的速度。针对这个最大最小化问题，利用等步长迭代算法即可求出最大的龙头

速度。

9.1.1 优化模型

目标函数： $\min V_{max}$

约束条件：

$$s.t.\begin{cases} V_{max} < 2m/s \\ \Gamma = constant \end{cases}$$

其中 V_0 是龙头的速度，即龙头速度不变；Γ 是“板凳龙”的运动轨迹，即“板凳龙”的运动路径确定。

决策变量：

该优化模型只有一个决策变量，即龙头速度 V_0。

9.2 模型求解

优化流程图如下：

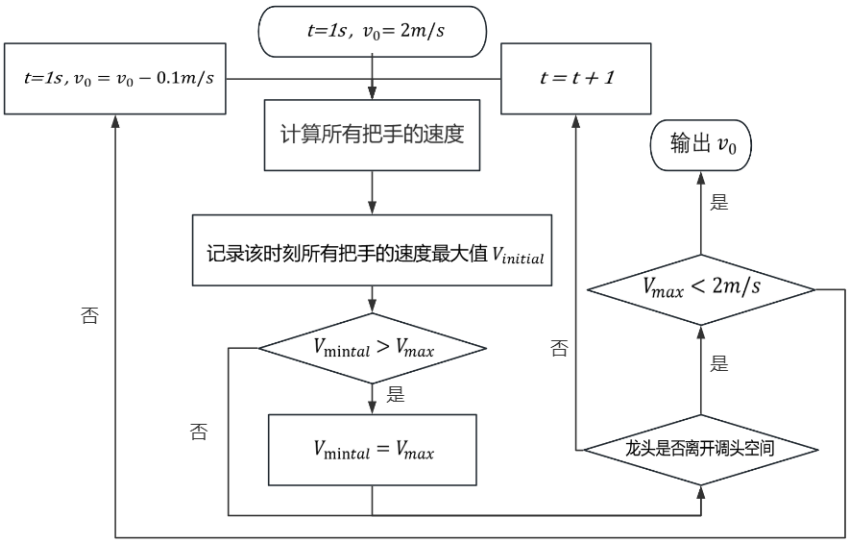


图 20　问题五流程图

Step1： 以龙头开始调头设为时间零点，以 1s 为时间步长,V_{max} 为把手速度最大值。

Step2： 时间前进一个步长，求解调头区域内所有把手的速度。

Step3： 将本次测试的所有把手的速度的最大值记为 V_{inital}

Step4： 若 $V_{inital} < V_{max}$ 则 $V_{max} = V_{max}$, 否则 $V_{max} = V_{inital}$

Step5： 当龙头离开调头空间后，结束循环，判定此时得到的 V_{max} 是否小于 $2m/s$

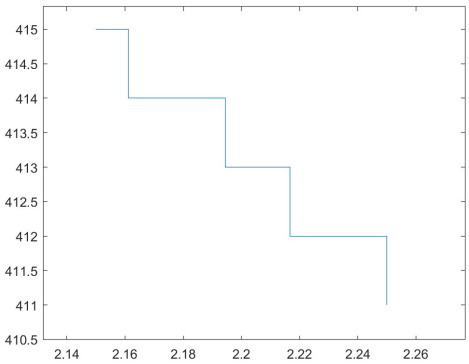
Step6： 若 Step5 输出为 True，则龙头的最大速度即为当前速度；若 Step5 输出为 Flase，则将龙头速度减小 $0.1m/s$，重复 Step1～ 5

9.3 求解结果

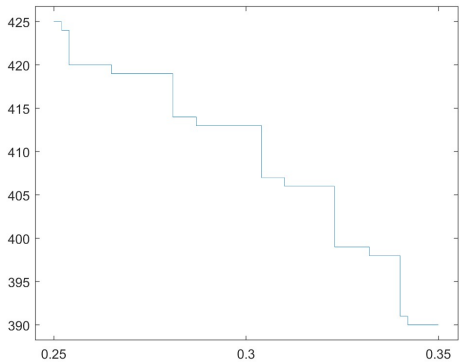
根据上述求解步骤，求出了最终的结果：当龙头速度为 $1.6m/s$，恰好“板凳龙”个把手速度均不会超过 2m/s。

十、 灵敏度分析

考虑到实际过程中，板凳龙的尺寸大小并不是严格等于给定尺寸，因此讨论板凳龙的尺寸大小对模型的灵敏度是十分必要的。这里假设板凳宽大小在 [0.25, 0.35] m 的范围内变化，取 0.001m 为步长，龙身长度在 [2.15, 2.25] m 的范围内变化，取 0.01m 为步长。计算问题二中的碰撞时间，结果如下：



(a) 长度灵敏度分析



(b) 宽度灵敏度分析

图 21　灵敏度分析图

可以发现当板凳的长度和宽度分别变化 0.1m 时，长度改变对终止时刻的影响更小，宽度改变对终止时刻的影响更大。所以板凳龙终止时间对板凳宽度更敏感。

十一、 模型的评价与推广

11.1 模型的优点

- 优点 1：本文对于速度的求解给出了递推公式，而不是使用差分法，这减小了我们模型的误差。
- 优点 2：本文在第三问使用二分法、第四问使用贝叶斯优化算法、第五问使用变步长迭代算法，针对不同的情况采用不同的优化策略，这极大程度上提高了我们的优化效率。

11.2 模型的缺点

- 缺点 1：本文采用的时间步长是 1s，得到的第二问的结果精度不高。同时这也对后面的优化的精度产生了一定的影响。

11.3 模型的推广

实际生活中，对于板凳龙的优化情况绝对不可能像题中给出的情况怎么简单，往往是多个决策变量同时对结果产生影响，而问题四中的贝叶斯优化对于这种多决策变量、观测函数计算代价极大的情况下的优势非常明显，可以利用极少的探测点得到较为精确的结果。面对多决策变量的情况，只需要在本文建立的优化模型的基础上添加决策变量即可，在这种情况下，贝叶斯优化的效率是远远高于一般的启发式算法的。

参考文献

[1] 刘娜毛晓菊. 基于分离轴定理的碰撞检测算法[J]. 电脑知识与技术, 2012, 1007(9416): 08-0102.

附录 A　文件列表

文件名	功能描述
q1 getPosition.m	问题一求解所有时刻所有把手位置坐标
q1 getVelosity.m	问题一计算所有时间点所有把手速度
q1 getXY.m	问题一计算所有时间点所有把手位置
q1 nextTheta.m	问题一运动一秒后的角度更新
q2 checkStatue.m	问题二检测碰撞函数
q2 findClosestAngles.m	问题二求解邻近层临近点
q2 getPosition.m	问题二求解所有时刻所有把手位置坐标
q2 getRmin.m	问题二计算碰撞时各参数
q2 getStopTime.m	问题二卡死时间
q2 getVelosity.m	问题二求解结果速度
q2 getXY.m	问题二求解结果坐标
q2 isCollision.m	问题二碰撞检测
q2 nextTheta.m	问题二一秒后角度
q2 plotEnd.m	问题二绘制龙头卡死状态
q3 checkStatue.m	问题三路径检测
q3 findClosestAngles.m	问题三求解最近的角度
q3 getDelta.m	问题三得到螺距
q3 getRmin.m	问题三得到龙头离中心的位置
q3 getStopTime.m	问题三结束时间
q3 isCollision.m	问题三程序代码
q3 nextTheta.m	问题三计算下一个角度
q3 plotResult.m	问题三结果画图程序
q3 solveBlackBox.m	问题三黑盒函数求救程序
q3 solveFunc.m	问题三程序代码
q3 test.m	问题三测试代码
q4 getVelocity.m	问题四程序代码
q4 getXY.m	问题四程序代码
q4 testGetAllPosition.m	问题四程序代码
q4 optBo.m	问题四程序代码
q4 checkHead.m	问题四程序代码
q4 getAllPosition.m	问题四程序代码

文件名	功能描述
q4 testGetAllVelocity.m	问题四程序代码
q4 getAllVelocity.m	问题四程序代码
q4 StoTau.m	问题四程序代码
q4 StoXY.m	问题四程序代码
q4 nextS.m	问题四程序代码
q4 getR.m	问题四程序代码
q4 testS2xy.m	问题四程序代码
q4 getHeadPosition.m	问题四程序代码
q4 plotTrail.m	问题四程序代码
q5 getAllPosition.m	问题五计算所有点自然坐标
q5 getAllVelocity.m	问题五速度获取程序
q5 getR.m	问题五计算圆弧轨迹半径
q5 nextS.m	问题五求解自然坐标方程
q5 objFunc.m	问题五目标函数
q5 OP5.m	问题五优化代码
q5 StoTau.m	问题五自然坐标到切向量的映射
q5 StoXY.m	问题五求解结果位置

附录 B　代码

q1 getPosition.m

```
clc,clear
close all

% 参数
pitch = 0.55;
velocity = 1;
time_total = 500;
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta, theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
    end
end

```

```
% 计算对应的半径 r
r_0 = lamnda * theta_0;

% 输出极坐标 (r, θ)
polar_positions = [r_0', theta_0'];

% 绘制螺旋线轨迹
%figure;
%polarplot(theta_0, r, 'r-', 'LineWidth', 1);
%title('龙头在螺旋线上的位置 (极坐标)');
%hold on
%polarplot(theta_0, r, 'lamnda*', 'LineWidth', 1);

%计算其余把手的位置
% 参数
L = 1.65*ones(1,223);  % 相邻把手之间的距离（米）
L(1) = 2.86;
num_handles = 224; % 把手总数

% 初始化每个把手的极坐标
r = zeros(224,time_total+1);
theta= zeros(224,time_total+1);

% 龙头的位置已知
r(1,:) = r_0;
theta(1,:) = theta_0;

% 根据余弦定理计算其余把手的位置
for j = 1:time_total+1
    for i = 2:num_handles
        theta_prev = theta(i-1,j);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1)); % 当前角度
        r_cur = lamnda * theta_cur; % 当前半径

        % 存储计算结果
```

```
        r(i,j) = r_cur;
        theta(i,j) = theta_cur;
    end
end

save("theta","theta");
save("r","r");
```

q1 getVelosity.m

```
%求解速度的递推方程
clear,clc;
%相关常量和数据
lammda=55/(200*pi);%螺旋线参数
v1=1;%第一个锚点线速度
numBench=223;%板凳个数
numTime=300 + 1;
numPoint=numBench+1;%锚点个数
load("theta.mat");
load("r.mat");

%正余弦矩阵
sin_theta=sin(theta);
cos_theta=cos(theta);

%龙头约束
V_theta=zeros(numPoint,numTime);%一个224*301的矩阵
V_theta(1,:)=v1./(lammda*sqrt(1+theta(1,:).^2));

%转移系数矩阵
transferCoefficient=zeros(numPoint-1,numTime);%一个(224-1)*301
    的矩阵
for j=1:numTime
    for i=1:numPoint-1
        transferCoefficient(i,j)=((r(i,j)*cos_theta(i,j)-r(i
   +1,j)*cos_theta(i+1,j))*(lammda*cos_theta(i,j)-r(i,j)*
```

```
sin_theta(i,j))+(r(i,j)*sin_theta(i,j)-r(i+1,j)*sin_theta(i
+1,j))*(r(i,j)*cos_theta(i,j)+lammda*sin_theta(i,j)))/((r(i,
j)*sin_theta(i,j)-r(i+1,j)*sin_theta(i+1,j))*(lammda*
sin_theta(i+1,j)+r(i+1,j)*cos_theta(i+1,j))-(r(i,j)*
cos_theta(i,j)-r(i+1,j)*cos_theta(i+1,j))*(r(i+1,j)*
sin_theta(i+1,j)-lammda*cos_theta(i+1,j)));
    end
end

%角速度递推
for j=1:numTime
    for i=1:numPoint-1
        V_theta(i+1,:)=transferCoefficient(i,:).*V_theta(i,:);
    end
end

%计算线速度
V=lammda*V_theta.*sqrt(1+theta.^2);%一个224*301的矩阵
save("V","V");
```

q1 getXY.m

```
clc,clear

load("r.mat")
load("theta.mat")

%初始化xy轴坐标
x = zeros(224,301);
y = zeros(224,301);

%计算xy轴坐标
for i = 1:224
    for j = 1:301
        x_cur = r(i,j) * cos(theta(i,j));
        y_cur = r(i,j) * sin(theta(i,j));
```

```
        x(i,j) = x_cur;
        y(i,j) = y_cur;
    end
end

save("x","x");
save("y","y");
```

q1 nextTheta.m

```
function val = nextTheta(theta_prev, l)
    % 参数
    b = 0.55 / (2 * pi);      % 螺旋线的系数
    r1 = b * theta_prev;      % 当前的半径

    % 定义目标函数
    fun = @(theta_cur) sqrt(r1^2 + (b * theta_cur)^2 - 2 * r1 * (b * theta_cur) * cos(theta_cur - theta_prev)) - l;

    % 初始猜测值
    x0 = theta_prev;

    % 使用fsolve求解
    options = optimoptions('fsolve', 'Display', 'off'); % 禁止显示求解过程
    [val, ~] = fsolve(fun, x0, options);

end
```

q2 checkStatue.m

```
function statue = checkStatue(statue_theta)
%接收某个时刻的theta状态向量，返回是否碰撞
near_angles=findClosestAngles(statue_theta);

statue=false;
for i=1:length(statue_theta)-30 %预留一些不参与主动检测，只参与被动检测
```

```
    if statue_theta(i)>(2.5*pi+statue_theta(1))
        break;
    end
    rect1_theta=[statue_theta(i);statue_theta(i+1)];
    rect2_theta=[statue_theta(near_angles(i,4));statue_theta(near_angles(i,4)+1)];
    if isCollision(rect1_theta,rect2_theta)
        statue=true;
        break;
    end
    rect2_theta=[statue_theta(near_angles(i,5));statue_theta(near_angles(i,5)+1)];
    if isCollision(rect1_theta,rect2_theta)
        statue=true;
        break;
    end
end
```

q2 findClosestAngles.m

```
function result = findClosestAngles(angles)
    % 输入:
    %   angles: 列向量，表示弧度制角度
    %
    % 输出:
    %   result: 每个角度及其最接近的两个角度以及它们的索引位置

    n = length(angles);
    result = zeros(n, 5);  % 创建一个结果矩阵，包含原角度、两个最邻近角度及其索引

    % 对原始角度进行排序，并保留原始索引
    [sorted_angles, original_indices] = sort(angles);

    % 对于每个角度，找到它的最接近的两个角度
    for i = 1:n
```

```
        angle = angles(i);
        shifted_angle = angle + 2 * pi;  % 当前角度加上2*pi

        % 查找比 shifted_angle 大的第一个元素
        idx = find(sorted_angles > shifted_angle, 1);

        % 如果不存在比 shifted_angle 大的元素，说明角度绕回到起点
        if isempty(idx)
            idx = n;
        end

        % idx 和 idx-1 就是最接近 shifted_angle 的两个元素
        if idx == 1
            closest1 = sorted_angles(1);
            closest2 = sorted_angles(2);
            idx1 = original_indices(1);
            idx2 = original_indices(2);
        elseif idx == n
            closest1 = sorted_angles(n-1);
            closest2 = sorted_angles(n);
            idx1 = original_indices(n-1);
            idx2 = original_indices(n);
        else
            closest1 = sorted_angles(idx-1);
            closest2 = sorted_angles(idx);
            idx1 = original_indices(idx-1);
            idx2 = original_indices(idx);
        end

        % 将结果存储到矩阵中（角度，最近的两个角度，这两个角度的原始索引）
        result(i, :) = [angle, closest1, closest2, idx1, idx2];
    end
```

```
end
```

q2 getPosition.m

```
clc,clear
close all

% 参数
pitch = 0.55;
velocity = 1;
time_total = getStopTime();
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta, theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
    end
end

```

```

%计算其余把手的位置
% 参数
num_handles = 50; % 只考虑前50个把手
L = 1.65*ones(1,num_handles-1);  % 相邻把手之间的距离（米）
L(1) = 2.86;

% 初始化每个把手的极坐标
theta= zeros(num_handles,time_total+1);

% 龙头的位置已知
theta(1,:) = theta_0;

% 根据余弦定理计算其余把手的位置
for j = 1:time_total+1
    for i = 2:num_handles
        theta_prev = theta(i-1,j);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1)); % 当前角度

        % 存储计算结果
        theta(i,j) = theta_cur;
    end
end
save("theta","theta");
```

q2 getRmin.m

```
%计算初次碰撞时刻

clc,clear
close all

% 参数
pitch = 0.55;
velocity = 1;
time_total = getStopTime();
```

```
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta,
   theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
    end
end


%计算其余把手的位置
% 参数
num_handles = 50; % 只考虑前50个把手
L = 1.65*ones(1,num_handles-1+20);  % 相邻把手之间的距离（米）
L(1) = 2.86;

% 初始化每个把手的极坐标
theta= zeros(num_handles+20,time_total+1);

```

```
% 龙头的位置已知
theta(1,:) = theta_0;

% 根据余弦定理计算其余把手的位置
for j = 1:time_total+1
    for i = 2:num_handles+20%多算一些保证余量
        theta_prev = theta(i-1,j);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1)); % 当前角度

        % 存储计算结果
        theta(i,j) = theta_cur;
    end
end

result = 1;
start_time = 0; % 初始的开始时间
increment = 50; % 每次递增的步长

while start_time <= time_total
    try
        for i = start_time:time_total
            if checkStatue(theta(:,i))
                result = i - 1;
                break;
            end
        end
        break; % 如果成功执行则跳出循环
    catch ME
        if strcmp(ME.identifier, 'MATLAB:badsubscript')
            % 如果捕捉到数组越界错误，增加start_time
            start_time = start_time + increment;
            if start_time > time_total
                error('已超出time_total，无法继续');
            end
        else
```

```
            % 其他错误则重新抛出
            rethrow(ME);
        end
    end
end

%停止时龙头举原点距离
r_min=lamnda*theta(1,result);
save("r_min","r_min");
save("result","result");%保存最终结果，如果result=1说明不会发生碰撞
```

q2 getStopTime.m

```
function [stopTime,lastTheta] = getStopTime()
%输出终止时刻和最后的龙头位置

% 参数
pitch = 0.55;
time_total = 1000;%足够长的时间
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);
theta_min=2.86*pi/pitch-0.5*pi;%龙头的最小方位角

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
```

```
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta,
   theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
        if theta_0(i)<theta_min
            stopTime=i-1;
            break;
        end
    end
end

lastTheta=theta_0(stopTime);
```

q2 getVelosity.m

```
%求解速度的递推方程
clear,clc;
%相关常量和数据
lammda=55/(200*pi);%螺旋线参数
v1=1;%第一个锚点线速度
numBench=223;%板凳个数
numTime=1;
numPoint=numBench+1;%锚点个数
load("theta.mat");

r=lammda*theta;
%正余弦矩阵
sin_theta=sin(theta);
cos_theta=cos(theta);

%龙头约束
V_theta=zeros(numPoint,numTime);%一个224*1的矩阵
V_theta(1,:)=v1./(lammda*sqrt(1+theta(1,:).^2));
```

```
%转移系数矩阵
transferCoefficient=zeros(numPoint-1,numTime);%一个(224-1)*1的矩阵
for j=1:numTime
    for i=1:numPoint-1
        transferCoefficient(i,j)=((r(i,j)*cos_theta(i,j)-r(i+1,j)*cos_theta(i+1,j))*(lammda*cos_theta(i,j)-r(i,j)*sin_theta(i,j))+(r(i,j)*sin_theta(i,j)-r(i+1,j)*sin_theta(i+1,j))*(r(i,j)*cos_theta(i,j)+lammda*sin_theta(i,j)))/((r(i,j)*sin_theta(i,j)-r(i+1,j)*sin_theta(i+1,j))*(lammda*sin_theta(i+1,j)+r(i+1,j)*cos_theta(i+1,j))-(r(i,j)*cos_theta(i,j)-r(i+1,j)*cos_theta(i+1,j))*(r(i+1,j)*sin_theta(i+1,j)-lammda*cos_theta(i+1,j)));
    end
end

%角速度递推
for j=1:numTime
    for i=1:numPoint-1
        V_theta(i+1,:)=transferCoefficient(i,:).*V_theta(i,:);
    end
end

%计算线速度
V=lammda*V_theta.*sqrt(1+theta.^2);%一个224*1的矩阵
save("V","V");
```

q2 getXY.m

```
clc,clear
close all

% 参数
pitch = 0.55;
velocity = 1;
```

```
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);

load("r_min");

%计算其余把手的位置
% 参数
num_handles = 224;
L = 1.65*ones(1,num_handles-1);  % 相邻把手之间的距离（米）
L(1) = 2.86;

% 初始化每个把手的极坐标
theta= zeros(num_handles,1);

% 龙头的位置已知
theta(1) = r_min/lamnda;

% 根据余弦定理计算其余把手的位置
for i = 2:num_handles
    theta_prev = theta(i-1);%前一个把手的角度
    theta_cur = nextTheta(theta_prev,L(i-1)); % 当前角度

    % 存储计算结果
    theta(i) = theta_cur;
end

save("theta","theta");
%初始化xy轴坐标
x = zeros(224,1);
y = zeros(224,1);

%计算xy轴坐标
for i = 1:224
    x_cur = lamnda*theta(i) * cos(theta(i));
```

```
    y_cur = lamnda*theta(i) * sin(theta(i));
    x(i) = x_cur;
    y(i) = y_cur;
end

save("x","x");
save("y","y");
```

q2 isCollision.m

```
%SAT碰撞检测
function isCollision = isCollision(rect1_theta, rect2_theta)
    % 检查两个旋转矩形是否发生碰撞
    %
    % 输入参数:
    %   rect1_polar: 第一个矩形的两个把手 (1x2矩阵, 格式为 [theta1;theta2])
    %   theta1应当小于theta2
    %   rect2_polar: 第二个矩形的两个把手 (1x2矩阵)
    %
    % 输出参数:
    %   isCollision: 如果两个矩形发生碰撞, 则返回 true; 否则返回 false

    lamnda=0.55/(2*pi);

    %极坐标矩阵
    rect1_polar=[rect1_theta*lamnda,rect1_theta];
    rect2_polar=[rect2_theta*lamnda,rect2_theta];

    %合成四点矩阵
    rect1_handle=rect1_polar(:,1).*[cos(rect1_polar(:,2)),sin(rect1_polar(:,2))];%转化为直角坐标
    rect2_handle=rect2_polar(:,1).*[cos(rect2_polar(:,2)),sin(rect2_polar(:,2))];%转化为直角坐标
    s1=rect1_handle(1,:)-rect1_handle(2,:);s1=s1/sqrt(sum(s1
```

```
.^2));%矩形一把手相对位矢
 n1=[-s1(2),s1(1)];%法向矢量
 s2=rect2_handle(1,:)-rect2_handle(2,:);s2=s2/sqrt(sum(s2
.^2));%矩形二把手相对位矢
 n2=[-s2(2),s2(1)];%法向矢量
 rect1_points = [rect1_handle(1,:)+0.275*s1+0.15*n1;
                 rect1_handle(1,:)+0.275*s1-0.15*n1;
                 rect1_handle(2,:)-0.275*s1-0.15*n1;
                 rect1_handle(2,:)-0.275*s1+0.15*n1];%矩形1
的四点矩阵
 rect2_points = [rect2_handle(1,:)+0.275*s2+0.15*n2;
                 rect2_handle(1,:)+0.275*s2-0.15*n2;
                 rect2_handle(2,:)-0.275*s2-0.15*n2;
                 rect2_handle(2,:)-0.275*s2+0.15*n2];%矩形2
的四点矩阵

 % 将两个法向量组合成一个单元数组，便于遍历
 normals = {n1,n2};

 % 投影并判断碰撞
 for j = 1:2

     % 将两个矩形的顶点投影到法向量上
     projection1 = projectRectangle(rect1_points, normals{j
});
     projection2 = projectRectangle(rect2_points, normals{j
});

     % 检查投影是否有重叠
     if ~overlap(projection1, projection2)
         % 如果在某个法向量上没有重叠，说明两个矩形不碰撞
         isCollision = false;
         return;
     end
 end
```

```
    % 如果所有法向量上的投影都重叠，则两个矩形碰撞
    isCollision = true;
end



function proj = projectRectangle(rect_points, axis)
    % projectRectangle: 将矩形的所有顶点投影到指定轴上
    %
    % 输入参数:
    %   rect_points: 矩形的顶点 (2x4矩阵)
    %   axis: 投影的轴向量
    %
    % 输出参数:
    %   proj: 投影后的最小值和最大值 [min_proj, max_proj]

    projections = rect_points * axis';  % 点乘，计算每个顶点在轴上的投影
    proj = [min(projections), max(projections)];
end

function isOverlap = overlap(projection1, projection2)
    % overlap: 检查两个投影是否有重叠
    %
    % 输入参数:
    %   projection1: 第一个投影的范围 [min1, max1]
    %   projection2: 第二个投影的范围 [min2, max2]
    %
    % 输出参数:
    %   isOverlap: 如果两个投影有重叠，则返回 true；否则返回 false

    isOverlap = ~(projection1(2) < projection2(1) || projection1(1) > projection2(2));
```

```
end
```

q2 nextTheta.m

```
function val = nextTheta(theta_prev, l)
    % 参数
    b = 0.55 / (2 * pi);      % 螺旋线的系数
    r1 = b * theta_prev;      % 当前的半径

    % 定义目标函数
    fun = @(theta_cur) sqrt(r1^2 + (b * theta_cur)^2 - 2 * r1 * (b * theta_cur) * cos(theta_cur - theta_prev)) - l;

    % 初始猜测值
    x0 = theta_prev;

    % 使用fsolve求解
    options = optimoptions('fsolve', 'Display', 'off'); % 禁止显示求解过程
    [val, ~] = fsolve(fun, x0, options);

end
```

q2 plotEnd.m

```
%绘制终止时刻示意图
clear,clc;

% 参数
pitch = 0.55;
lamnda = pitch / (2 * pi);
L = 1.65*ones(1,223);  % 相邻把手之间的距离（米）
L(1) = 2.86;
num_handles = 224; % 把手总数

stop_theta=zeros(num_handles,1);

[stop_time,stop_theta(1)]=getStopTime();
```

```
for i = 2:num_handles
        theta_prev = stop_theta(i-1);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1)); % 当前角度

        % 存储计算结果
        stop_theta(i) = theta_cur;
end
stop_r=lamnda*stop_theta;

% 绘制折线图
figure;
polarplot(stop_theta, stop_r, 'r-', 'LineWidth', 1);
% title('最终时刻的把手图形');
hold on
%绘制螺旋线
sample_theta=0:0.01:100;
sample_r=lamnda*sample_theta;
polarplot(sample_theta,sample_r,'b-','LineWidth',1);
```

q3 checkStatue.m

```
function statue = checkStatue(statue_theta,pitch)
%接收某个时刻的theta状态向量，返回是否碰撞
near_angles=findClosestAngles(statue_theta);

statue=false;
for i=1:length(statue_theta)-30 %留下一些外圈的点不参与主动检测，只参与被动检测
    if statue_theta(i)>(2.5*pi+statue_theta(1))
        break;
    end
    rect1_theta=[statue_theta(i);statue_theta(i+1)];
    rect2_theta=[statue_theta(near_angles(i,4));statue_theta(near_angles(i,4)+1)];
    if isCollision(rect1_theta,rect2_theta,pitch)
        statue=true;
```

```
        break;
    end
    rect2_theta=[statue_theta(near_angles(i,5));statue_theta(near_angles(i,5)+1)];
    if isCollision(rect1_theta,rect2_theta,pitch)
        statue=true;
        break;
    end
end
```

q3 findClosestAngles.m

```
function result = findClosestAngles(angles)
    % 输入:
    %   angles: 列向量, 表示弧度制角度
    %
    % 输出:
    %   result: 每个角度及其最接近的两个角度以及它们的索引位置

    n = length(angles);
    result = zeros(n, 5);  % 创建一个结果矩阵, 包含原角度、两个最邻近角度及其索引

    % 对原始角度进行排序, 并保留原始索引
    [sorted_angles, original_indices] = sort(angles);

    % 对于每个角度, 找到它的最接近的两个角度
    for i = 1:n
        angle = angles(i);
        shifted_angle = angle + 2 * pi;  % 当前角度加上2*pi

        % 查找比 shifted_angle 大的第一个元素
        idx = find(sorted_angles > shifted_angle, 1);

        % 如果不存在比 shifted_angle 大的元素, 说明角度绕回到起点
```

```
        if isempty(idx)
            idx = n;
        end

        % idx 和 idx-1 就是最接近 shifted_angle 的两个元素
        if idx == 1
            closest1 = sorted_angles(1);
            closest2 = sorted_angles(2);
            idx1 = original_indices(1);
            idx2 = original_indices(2);
        elseif idx == n
            closest1 = sorted_angles(n-1);
            closest2 = sorted_angles(n);
            idx1 = original_indices(n-1);
            idx2 = original_indices(n);
        else
            closest1 = sorted_angles(idx-1);
            closest2 = sorted_angles(idx);
            idx1 = original_indices(idx-1);
            idx2 = original_indices(idx);
        end

        % 将结果存储到矩阵中（角度，最近的两个角度，这两个角度的原始索引）
        result(i, :) = [angle, closest1, closest2, idx1, idx2];
    end
end
```

q3 getDelta.m

```
clear,clc;

sample_points=linspace(0.44972,0.44973,20);
r_min=zeros(1,30);
for i=1:30
```

```
    r_min(i)=getRmin(sample_points(i));
end
plot(sample_points,r_min);
save("r_min","r_min");

%%
clear,clc;

load("r_min.mat");
sample_points=linspace(0.4,0.5,30);

%曲线拟合并调优
p=polyfit(sample_points,r_min,5);
hold on;
plot(sample_points,r_min,'.');
more_x=linspace(sample_points(1),sample_points(end),100);
plot(more_x,polyval(p,more_x));

%求解零点
p(end)=p(end)-4.5;
delta_mins=roots(p);
% 指定区间
interval = [sample_points(1), 0.45];
% interval = [0.447, sample_points(end)];

% 筛选出在指定区间内的零点
delta_min = delta_mins(delta_mins >= interval(1) & delta_mins
   <= interval(2));
plot(delta_min,4.5,'o');
```

q3 getRmin.m

```
function [r_min,result] = getRmin(pitch)
%计算初次碰撞时刻龙头距离原点的距离

% 参数
```

```
time_total = getStopTime(pitch);
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta, theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
    end
end


%计算其余把手的位置
% 参数
num_handles = 50; % 只考虑前50个把手
L = 1.65*ones(1,num_handles-1+30);  % 相邻把手之间的距离（米），+30是保持第50个能够访问到其对应外圈
L(1) = 2.86;

% 初始化每个把手的极坐标
```

```
theta= zeros(num_handles+30,time_total+1);

% 龙头的位置已知
theta(1,:) = theta_0;

% 根据余弦定理计算其余把手的位置
for j = 1:time_total+1
    for i = 2:num_handles+30%多算一些保证余量
        theta_prev = theta(i-1,j);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1),pitch); % 当前角度

        % 存储计算结果
        theta(i,j) = theta_cur;
    end
end

result = 1;
start_time = 0; % 初始的开始时间
increment = 50; % 每次递增的步长

while start_time <= time_total
    try
        for i = start_time:time_total
            if checkStatue(theta(:,i), pitch)
                result = i - 1;
                break;
            end
        end
        break; % 如果成功执行则跳出循环
    catch ME
        if strcmp(ME.identifier, 'MATLAB:badsubscript')
            % 如果捕捉到数组越界错误，增加start_time
            start_time = start_time + increment;
            if start_time > time_total
```

```
                error('已超出time_total，无法继续');
            end
        else
            % 其他错误则重新抛出
            rethrow(ME);
        end
    end
end

%停止时龙头距原点距离
r_min=lamnda*theta(1,result);
%如果最终结果result=1说明龙头卡死时都没有发生碰撞
```

q3 getStopTime.m

```
function [stopTime,lastTheta] = getStopTime(pitch)
%输出终止时刻和最后的龙头位置

% 参数
time_total = 10000;%足够长的时间
time_step = 1;
initial_theta = 32 * pi;
lamnda = pitch / (2 * pi);
theta_min=2.86*pi/pitch-0.5*pi;%龙头的最小方位角

%时间步长
time = 0:time_step:time_total;

% 初始化theta表格
theta_0 = zeros(size(time));

% 为每个弧长 s 求解对应的 θ
for i = 1:length(time)
    if i==1
        theta0=initial_theta;
        theta_0(i)=theta0;
```

```
    else
        theta0=theta_0(i-1);
            s_target = -1;
        ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
        theta_change = fzero(@(theta1) integral(ds_dtheta, theta0, theta1) - s_target, theta0);
        theta_0(i)=theta_change;
        if theta_0(i)<theta_min
            stopTime=i-1;
            lastTheta=theta_0(stopTime);
            break;
        end
    end
end
```

q3 isCollision.m

```
%SAT碰撞检测
function isCollision = isCollision(rect1_theta, rect2_theta, pitch)
    % 检查两个旋转矩形是否发生碰撞
    %
    % 输入参数:
    %   rect1_polar: 第一个矩形的两个把手 (1x2矩阵, 格式为 [theta1;theta2])
    %   theta1应当小于theta2
    %   rect2_polar: 第二个矩形的两个把手 (1x2矩阵)
    %
    % 输出参数:
    %   isCollision: 如果两个矩形发生碰撞, 则返回 true; 否则返回 false

    lamnda=pitch/(2*pi);

    %极坐标矩阵
    rect1_polar=[rect1_theta*lamnda,rect1_theta];
```

```
 rect2_polar=[rect2_theta*lamnda,rect2_theta];

 %合成四点矩阵
 rect1_handle=rect1_polar(:,1).*[cos(rect1_polar(:,2)),sin(rect1_polar(:,2))];%转化为直角坐标
 rect2_handle=rect2_polar(:,1).*[cos(rect2_polar(:,2)),sin(rect2_polar(:,2))];%转化为直角坐标
 s1=rect1_handle(1,:)-rect1_handle(2,:);s1=s1/sqrt(sum(s1.^2));%矩形一把手相对位矢
 n1=[-s1(2),s1(1)];%法向矢量
 s2=rect2_handle(1,:)-rect2_handle(2,:);s2=s2/sqrt(sum(s2.^2));%矩形二把手相对位矢
 n2=[-s2(2),s2(1)];%法向矢量
 rect1_points = [rect1_handle(1,:)+0.275*s1+0.15*n1;
                 rect1_handle(1,:)+0.275*s1-0.15*n1;
                 rect1_handle(2,:)-0.275*s1-0.15*n1;
                 rect1_handle(2,:)-0.275*s1+0.15*n1];%矩形1的四点矩阵
 rect2_points = [rect2_handle(1,:)+0.275*s2+0.15*n2;
                 rect2_handle(1,:)+0.275*s2-0.15*n2;
                 rect2_handle(2,:)-0.275*s2-0.15*n2;
                 rect2_handle(2,:)-0.275*s2+0.15*n2];%矩形2的四点矩阵

 % 将两个法向量组合成一个单元数组，便于遍历
 normals = {n1,n2};

 % 投影并判断碰撞
 for j = 1:2

     % 将两个矩形的顶点投影到法向量上
     projection1 = projectRectangle(rect1_points, normals{j});
     projection2 = projectRectangle(rect2_points, normals{j});
```

```
        % 检查投影是否有重叠
        if ~overlap(projection1, projection2)
            % 如果在某个法向量上没有重叠，说明两个矩形不碰撞
            isCollision = false;
            return;
        end
    end

    % 如果所有法向量上的投影都重叠，则两个矩形碰撞
    isCollision = true;
end



function proj = projectRectangle(rect_points, axis)
    % projectRectangle: 将矩形的所有顶点投影到指定轴上
    %
    % 输入参数:
    %   rect_points: 矩形的顶点 (2x4矩阵)
    %   axis: 投影的轴向量
    %
    % 输出参数:
    %   proj: 投影后的最小值和最大值 [min_proj, max_proj]

    projections = rect_points * axis';  % 点乘，计算每个顶点在轴上的投影
    proj = [min(projections), max(projections)];
end

function isOverlap = overlap(projection1, projection2)
    % overlap: 检查两个投影是否有重叠
    %
    % 输入参数:
    %   projection1: 第一个投影的范围 [min1, max1]
```

```
    %   projection2: 第二个投影的范围 [min2, max2]
    %
    % 输出参数:
    %   isOverlap: 如果两个投影有重叠, 则返回 true; 否则返回 false

    isOverlap = ~(projection1(2) < projection2(1) || projection1(1) > projection2(2));
end
```

q3 nextTheta.m

```
function val = nextTheta(theta_prev, l,pitch)
    % 参数
    b = pitch / (2 * pi);      % 螺旋线的系数
    r1 = b * theta_prev;      % 当前的半径

    % 定义目标函数
    fun = @(theta_cur) sqrt(r1^2 + (b * theta_cur)^2 - 2 * r1 * (b * theta_cur) * cos(theta_cur - theta_prev)) - l;

    % 初始猜测值
    x0 = theta_prev;

    % 使用fsolve求解
    options = optimoptions('fsolve', 'Display', 'off'); % 禁止显示求解过程
    [val, ~] = fsolve(fun, x0, options);

end
```

q3 plotResult.m

```
%绘制终止时刻示意图
clear,clc;

% 参数
load("delta_min.mat");
```

```
pitch = delta_min;
lamnda = pitch / (2 * pi);
L = 1.65*ones(1,223);  % 相邻把手之间的距离（米）
L(1) = 2.86;
num_handles = 224; % 把手总数

result_theta=zeros(num_handles,1);

[r_stop,~]=getRmin(pitch);
result_theta(1)=r_stop/lamnda;
for i = 2:num_handles
        theta_prev = result_theta(i-1);%前一个把手的角度
        theta_cur = nextTheta(theta_prev,L(i-1),pitch); % 当前角度

        % 存储计算结果
        result_theta(i) = theta_cur;
end
stop_r=lamnda*result_theta;

% 绘制折线图
figure;
polarplot(result_theta, stop_r, 'r-', 'LineWidth', 1);
% title('最终时刻的把手图形');
hold on
%绘制螺旋线
sample_theta=0:0.01:100;
sample_r=lamnda*sample_theta;
polarplot(sample_theta,sample_r,'b-','LineWidth',1);
```

q3 solveBlackBox.m

```
function [root,iterations] = solveBlackBox(func, a, b, tol)
    % 二分法求解黑盒函数

    % 检查初值是否满足零点存在条件
```

```
    if func(a) * func(b) >= 0
        error('func(a) 和 func(b) 必须异号');
    end

    % 初始化
    iterations = 0;
    maxIter=1000;
    while (b - a) / 2 > tol*0.01 && iterations < maxIter
        % 计算中点
        c = (a + b) / 2;

        % 更新迭代次数
        iterations = iterations + 1;

        % 判断是否找到零点
        if func(c) == 0
            break;
        elseif func(a) * func(c) < 0
            b = c;  % 零点在 [a, c] 区间
        else
            a = c;  % 零点在 [c, b] 区间
        end
    end

    % 最终根的位置
    root = (a + b) / 2;
end
```

q3 solveFunc.m

```
%更精确的求解

clear,clc;

f = @(x) deal(getRmin(x)) - 4.5;
[delta_min,iter_num]=solveBlackBox(f,0.44,0.45,1e-6);
```

```
% save("delta_min","delta_min");0.449719848632812
```

q3 test.m

```
% 绘制一个以4.5为半径的虚线边缘圆，并填充为灰色
clear; clc;

% 定义圆的半径和角度范围
radius = 4.5;
theta_circle = linspace(0, 2 * pi, 100);  % 0到2*pi的角度

% 生成圆的坐标
x_circle = radius * cos(theta_circle);  % 圆的x坐标
y_circle = radius * sin(theta_circle);  % 圆的y坐标

% 创建图形窗口
figure;

% 使用fill函数填充圆的内部为灰色
fill(x_circle, y_circle, [0.8 0.8 0.8], 'EdgeColor', 'none');
hold on;

% 绘制圆的虚线边缘
plot(x_circle, y_circle, 'k--', 'LineWidth', 1.5);

% 设置图形属性
axis equal;  % 使得x和y轴比例相等
xlabel('X轴');
ylabel('Y轴');
title('半径为4.5的圆，灰色填充，虚线边缘');

hold off;
```

q5 getAllPosition.m

```
function [xy_positions,s_positions] = getAllPosition(time_now,
   v1,theta1,theta2,num_handles)
%接受龙头的位置坐标，返回整个龙队所有把手的位置坐标
```

```

R=getR(theta1,theta2);

% num_handles=224;
L = 1.65*ones(1,num_handles-1);  % 相邻把手之间的距离（米）
L(1) = 2.86;

S_list=zeros(1,num_handles);
S_list(1)=time_now*v1;

for i=2:num_handles
    S_list(i)=nextS(S_list(i-1),L(i-1),v1,theta1,theta2,1e-2);
end
s_positions=S_list;

xy_positions(:,1)=StoXY(S_list(1),theta1,theta2,R);
for i=2:length(S_list)
xy_positions=[xy_positions;StoXY(S_list(i),theta1,theta2,R)];
end

end
```

q5 getAllVelocity.m

```
function V_all=getAllVelocity(s_list,xy_positions,v1,theta1,
   theta2,R,num_handles)
%接受某个时刻整个龙队的位置矩阵xy_positions和龙头的速度大小v1
   ，返回整个龙队的速度大小矩阵

% num_handles=224;

%板凳方向矩阵
near_rela=zeros(2,num_handles-1);
for i=1:num_handles-1
    near_rela(:,i)=[xy_positions(2*i-1)-xy_positions(2*i+1);
   xy_positions(2*i)-xy_positions(2*i+2)];
```

```
    near_rela(:,i)=near_rela(:,i)/norm(near_rela(:,i));%归一化
end

%把手切向矩阵
taus=zeros(2,num_handles);
for i=1:num_handles
    taus(:,i)=StoTau(s_list(i),theta1,theta2,R);
end

%转移系数矩阵
trans=zeros(1,num_handles-1);
for i=1:num_handles-1
    trans(i)=(near_rela(:,i)'*taus(:,i))/(near_rela(:,i)'*taus(:,1+i));
end

V_all=zeros(1,num_handles);
V_all(1)=v1;
for i=1:num_handles-1
V_all(i+1)=V_all(i)*trans(i);
end
```

q5 getR.m

```
function [val,n1,n2,r1,r2] = getR(theta1,theta2)
%接受两个角度参数，返回可行的小圆半径,如果无解则返回-1

lamnda=1.7/(2*pi);
%给定点坐标转化为直角系
r1=lamnda*theta1*[cos(theta1);sin(theta1)];
r2=lamnda*theta2*[cos(pi+theta2);sin(pi+theta2)];

%求解法向量并归一化
n1=[-sin(theta1)-theta1*cos(theta1);cos(theta1)-theta1*sin(theta1)];n1=n1/norm(n1);
n2=-[-sin(theta2)-theta2*cos(theta2);cos(theta2)-theta2*sin(
```

```
    theta2)];n2=n2/norm(n2);

%二次方程的系数
A=(norm(2*n1-n2))^2-9;
B=2*(r1-r2)'*(2*n1-n2);
C=norm(r1-r2)^2;

%判断条件
delta=B^2-4*A*C;
if delta<0
    val=-1;
    return;
end
if A==0
    val=-C/B;
    return;
end

val=(B+sqrt(B^2-4*A*C))/(-2*A);
return
```

q5 nextS.m

```
function next_s = nextS(S_now,obj_dis,v1,theta1,theta2,tol)

R=getR(theta1,theta2);

%递推约束方程
    function val=func(start_s,obj_dis,end_s,theta1,theta2,R)
        val=norm(StoXY(start_s,theta1,theta2,R)-StoXY(end_s,
    theta1,theta2,R))-obj_dis;
end

% 检查初值是否满足零点存在条件
time_test=3;
while true
```

```
if func(S_now,obj_dis,S_now-v1*time_test,theta1,theta2,R) * func(S_now,obj_dis,S_now-v1*0.5,theta1,theta2,R) >= 0
    time_test=time_test+1;
else
    break;
end
end

% 初始化
iterations = 0;
maxIter=1000;
a=S_now-v1*time_test;b=S_now-v1*0.5;
while (b - a) / 2 > tol && iterations < maxIter
    % 计算中点
    c = (a + b) / 2;

    % 更新迭代次数
    iterations = iterations + 1;

    % 判断是否找到零点
    if func(S_now,obj_dis,c,theta1,theta2,R) == 0
        break;
    elseif func(S_now,obj_dis,a,theta1,theta2,R) * func(S_now,obj_dis,c,theta1,theta2,R) < 0
        b = c;  % 零点在 [a, c] 区间
    else
        a = c;  % 零点在 [c, b] 区间
    end
end

% 最终根的位置
next_s = (a + b) / 2;
end
```

q5 objFunc.m

```
function val=objFunc(x)
%目标函数
theta1=x.x1;theta2=x.x2;

%理论约束
if theta2<theta1-pi || theta2>theta1+pi
    val=50;
    return;
end

%求解半径并检测
[R,n1,n2,r1,r2]=getR(theta1,theta2);
if R==-1
    val=50;
    return;
end

%求解圆心坐标
O1=2*R*n1+r1;
O2=R*n2+r2;
O1_O2=O2-O1;O1_O2=O1_O2/norm(O1_O2);%圆心间的相对位矢

%检测龙头卡死情况
time_total=100;%足够长的检测时间
isStick=checkHead(theta1,theta2,R,time_total,1);
if isStick
    val=50;
    return;
end

%计算符合实际的掉头路径长度
val=3*pi*R+R*(2*acos(n1'*O1_O2)+acos(n2'*O1_O2));
end
```

q5 OP5.m

```
clc, clear

% 参数设定
V0 = 1.9; % 龙头初始速度，单位m/s
Vmax = V0; % 初始最大速度，单位 m/s
Vstep = 0.1; % 每次降低的速度，单位 m/s
time_step = 1; % 时间步长，单位 s
head_out_of_turn = false; % 判断龙头是否离开调头区域
max_speed_limit = 2.0; % 速度上限
time_total = 100; % 检测时间上限
num_handles = 50; % 把手数目
Vinital = 0; % 当前所有把手速度的最大值
load("x_opt.mat"); % 上次优化结果
theta_in = x_opt(1); theta_out = x_opt(2);

R = getR(theta_in, theta_out);

while true

    % Step 1: 时间前进
    for t = 0:time_step:time_total

        % Step 2: 求解调头区域内所有把手的速度
        [xy_positions, s_list] = getAllPosition(t, V0, theta_in, theta_out, num_handles); % 当前时刻所有把手的位置
        V_all = getAllVelocity(s_list, xy_positions, V0, theta_in, theta_out, R, num_handles); % 当前时刻所有把手的速度

        % Step 3: 取所有把手的最大速度作为Vinital
        Vinital = max(V_all);

        % Step 4: 更新Vmax
        if Vinital >= Vmax
            Vmax = Vinital;
```

```
        end

        % Step 5: 判断是否离开调头区域
        if t > 10
            head_out_of_turn = true; % 标记为已经离开调头区域
            break; % 跳出for循环
        end
    end

    % Step 6: 判断最大速度是否小于2m/s
    if Vmax < max_speed_limit
        fprintf('龙头最大速度为 %.2f m/s\n', Vmax);
        break; % 满足条件，结束while循环
    else
        % 速度减小，重新计算
        V0 = V0 - Vstep;
        Vmax = V0; % 重置最大速度以重新计算
    end
end
```

q5 StoTau.m

```
function tau=StoTau(s,theta1,theta2,R)
%接收自然坐标s，返回该点处的切向量方向tau

lamnda=1.7/(2*pi);
%给定点坐标转化为直角系
r1=lamnda*theta1*[cos(theta1);sin(theta1)];
r2=lamnda*theta2*[cos(theta2+pi);sin(theta2+pi)];

%求解法向量并归一化
n1=[-sin(theta1)-theta1*cos(theta1);cos(theta1)-theta1*sin(
   theta1)];n1=n1/norm(n1);
n2=-[-sin(theta2)-theta2*cos(theta2);cos(theta2)-theta2*sin(
   theta2)];n2=n2/norm(n2);

```

```
%求解圆心坐标
O1=2*R*n1+r1;
O2=R*n2+r2;
O1_O2=O2-O1;O1_O2=O1_O2/norm(O1_O2);%圆心间的相对位矢

%求解顺时针的角度范围
v1 = r1-O1;  % 矢量1
v2 = O1_O2;  % 矢量2

alpha1 = atan2(v1(2), v1(1));
alpha2 = atan2(v2(2), v2(1));
xi1 = alpha1 - alpha2;
% 确保角度在0到2*pi之间
if xi1 < 0
    xi1 = xi1 + 2 * pi;
end

%求解逆时针的角度范围
v1 = -O1_O2;  % 矢量1
v2 = r2-O2;  % 矢量2

alpha1 = atan2(v1(2), v1(1));
alpha2 = atan2(v2(2), v2(1));
xi2 = alpha1 - alpha2;
% 确保角度在0到2*pi之间
if xi2 < 0
    xi2 = xi2 + 2 * pi;
end
xi2=2*pi-xi2;

if s<0
    ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
    theta_before = fzero(@(theta_next) integral(ds_dtheta,
   theta1, theta_next) + s, theta1);
    tau=[-cos(theta_before)+theta_before*sin(theta_before);-
```

```
sin(theta_before)-theta_before*cos(theta_before)];
    tau=tau/norm(tau);
    return;
end

if s<2*R*xi1
    change_theta1=s/(2*R);
    T1=[cos(change_theta1) sin(change_theta1);-sin(change_theta1) cos(change_theta1)];
    relative=T1*(r1-O1);
    tau=[relative(2);-relative(1)];
    tau=tau/norm(tau);
    return;
end

if s<(2*R*xi1+R*xi2)
    change_theta2=(s-2*R*xi1)/R;
    T2=[cos(change_theta2) -sin(change_theta2);sin(change_theta2) cos(change_theta2)];
    relative=T2*(-(O1_O2/norm(O1_O2))*R);
    tau=[-relative(2);relative(1)];
    tau=tau/norm(tau);
    return;
end

s_target = -(s-(2*R*xi1+R*xi2));
ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
theta_after = fzero(@(theta_next) integral(ds_dtheta, theta2, theta_next) + s_target, theta2);
tau=[-cos(theta_after)+theta_after*sin(theta_after);-sin(theta_after)-theta_after*cos(theta_after)];
tau=tau/norm(tau);
end
```

q5 StoXY.m

```
function position = StoXY(s,theta1,theta2,R)
%自然坐标与直角坐标的映射关系

lamnda=1.7/(2*pi);
%给定点坐标转化为直角系
r1=lamnda*theta1*[cos(theta1);sin(theta1)];
r2=lamnda*theta2*[cos(theta2+pi);sin(theta2+pi)];

%求解法向量并归一化
n1=[-sin(theta1)-theta1*cos(theta1);cos(theta1)-theta1*sin(theta1)];n1=n1/norm(n1);
n2=-[-sin(theta2)-theta2*cos(theta2);cos(theta2)-theta2*sin(theta2)];n2=n2/norm(n2);

%求解圆心坐标
O1=2*R*n1+r1;
O2=R*n2+r2;
O1_O2=O2-O1;O1_O2=O1_O2/norm(O1_O2);%圆心间的相对位矢

%求解顺时针的角度范围
v1 = r1-O1;  % 矢量1
v2 = O1_O2;  % 矢量2

alpha1 = atan2(v1(2), v1(1));
alpha2 = atan2(v2(2), v2(1));
xi1 = alpha1 - alpha2;
% 确保角度在0到2*pi之间
if xi1 < 0
    xi1 = xi1 + 2 * pi;
end

%求解逆时针的角度范围
v1 = -O1_O2;  % 矢量1
v2 = r2-O2;  % 矢量2

```

```
alpha1 = atan2(v1(2), v1(1));
alpha2 = atan2(v2(2), v2(1));
xi2 = alpha1 - alpha2;
% 确保角度在0到2*pi之间
if xi2 < 0
    xi2 = xi2 + 2 * pi;
end
xi2=2*pi-xi2;

if s<0
    ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
    theta_before = fzero(@(theta_next) integral(ds_dtheta,
   theta1, theta_next) + s, theta1);
    position=lamnda*theta_before*[cos(theta_before);sin(
   theta_before)];
    return;
end

if s<2*R*xi1
    change_theta1=s/(2*R);
    T1=[cos(change_theta1) sin(change_theta1);-sin(
   change_theta1) cos(change_theta1)];
    position=O1+T1*(r1-O1);
    return;
end

if s<(2*R*xi1+R*xi2)
    change_theta2=(s-2*R*xi1)/R;
    T2=[cos(change_theta2) -sin(change_theta2);sin(
   change_theta2) cos(change_theta2)];
    position=O2+T2*(-(O1_O2/norm(O1_O2))*R);
    return;
end

s_target = -(s-(2*R*xi1+R*xi2));
```

```
ds_dtheta = @(theta) lamnda * sqrt(theta.^2 + 1);
theta_after = fzero(@(theta_next) integral(ds_dtheta, theta2,
   theta_next) + s_target, theta2);
position=lamnda*(theta_after)*[cos(pi+theta_after);sin(pi+
   theta_after)];

end
```